QUICK START TO PYTHON

– Building on top of introductory programming experience

Contents

1 Moving from JavaScript / Java to Python 3
1.1 What is Python? 3
1.2 What software is needed to create and run Python programs? 3
1.3 Scripts and modules, running a Python program 4
1.4 Comparing a Python program to a JavaScript program and a Java program 6
2 Program structure 7
2.1 First Python program examples 7
2.2 Python naming convention 8
2.3 Symbols in a Python source text, whitespace, comments, docstrings 8
2.4 Maximum code line length / How to split one line of code over two or more lines 9
3 Printing to the console, formatting output 10
4 Inputting data from the user 13
4.1 Inputting data 13
4.2 Handling invalid input | Exception handling 13
5 Variables and data types 15
5.1 Overview 15
5.2 String type (str), string literals 16
5.3 Boolean type (bool), truthy and falsy values 17
5.4 Explicit type conversions 17
5.5 Scopes of variables 18
6 Basic operators and expressions, mathematical calculations 20
6.1 Operators and expressions 20
6.2 Functions for mathematical computations 21
6.3 Type conversions 22
6.4 Inaccurate floating-point values, using Decimal for precise decimal values 23
7 Selection (if statement) 25
8 Repetition (while loop, for loop) 26
8.1 While loop 26
8.2 For loop 26
9 Defining functions 29
9.1 Function definitions 29
9.2 Passing arguments to functions 30
9.3 Using type hints in a function definition 33
9.4 Using the pass statement for a temporarily placeholder of required syntax 34
9.5 Why create and use functions? 34
9.6 Distributing reusable functions in a library module 35
9.7 Conditional execution of the main function 36
9.8 How to write simple unit tests with doctest 37

10 List handling 38
10.1 Basic operations 38
10.2 Using set() to remove duplicates from a list 40
10.3 Comprehensive list of list operations 41
10.4 List comprehension 43
11 String handling 44
11.1 Overview 44
11.2 String handling with the methods of the str class 45
11.3 Extracting parts of strings with indexing and slicing 47
11.4 Splitting and joining strings 48
12 More list handling – Matrices 49
12.1 Creating and manipulating a matrix 49
12.2 Modifying items in a matrix 50
12.3 Visualising a Python program with the Python Tutor tool 51
13 References 52
13.1 Python handles all data items as objects 52
13.2 How to make a copy of a list or a collection of another type 52
13.3 Loop counting is slower in Python (*for advanced learners*) 54
14 Short overview of collection types (list, tuple, set, dictionary) 55
15 Tuples and sequences 56
15.1 Sequences (common features, comparison operations) 56
15.2 Tuples 56
15.3 Named tuples 59
17 Dictionaries 60
18 Reading text files 64
18.1 Reading the content of a text file 64
18.2 Using Path to locate the absolute path to a folder 65
18.3 Reading a CSV file 66
List of Python keywords used in the examples 67

Note: *Object-oriented programming* is not covered in this guide.

The Python Reference

Style Guide for Python Code:	https://peps.python.org/pep-0008	😀
The Python Standard Library:	https://docs.python.org/3/library/index.html	😀
The Python Language Reference:	https://docs.python.org/3/reference/index.html	🌶️
The Full Python Grammar:	https://docs.python.org/3/reference/grammar.html	🌶️🌶️🌶️
The Python Tutorial:	https://docs.python.org/3/tutorial/index.html	🌶️

Other sources

Wikipedia:	https://en.wikipedia.org/wiki/Python_(programming_language)
Python Books:	https://wiki.python.org/moin/PythonBooks

Software

Python download page	https://www.python.org/downloads
Python Package Index	https://pypi.org
VS Code download page	https://code.visualstudio.com/Download

1 Moving from JavaScript / Java to Python

1.1 What is Python?

Python is multi-paradigm, multi-purpose programming language. It fully supports object-oriented and procedural programming. In addition, it offers some support to functional programming and some other programming paradigms. Python is an interpreted, dynamically-typed programming language where all data is represented as objects. The most recent major version of Python is Python 3[1].

The core language is small, but the standard library is large, and the language is easily extensible via modules. The standard library has an especially rich set of mathematical and statistical tools. There are a lot of popular third-party libraries that are useful in data analysis and data visualisation, machine learning, web development etc.

Python, Java, and JavaScript have quite different origins.

- The first version **Java** was designed in 1991 for intelligent consumer-electronic devices (interactive television etc.) by a team lead by James Gosling at Sun Microsystems. The current developer of the language is Oracle Corporation.
- The first version of **JavaScript** (originally named *Mocha*, then *LiveScript*) was created in 1995 by Brendan Eich at Netscape "to control almost everything in the Netscape [web] browser".
- The first version of **Python** (named after the Monty Python's Flying Circus, a BBC comedy series from the 1970s) was designed in 1990 by Guido van Rossum at Centrum Wiskunde & Informatica in the Netherlands. The current developer of the language is Python Software Foundation.

See also https://en.wikipedia.org/wiki/Python_(programming_language)

1.2 What software is needed to create and run Python programs?

1. You should install a **Python implementation** on your computer. A Python implementation means environment which provides support for the execution of programs written in the Python language. The environment always includes the **Python interpreter** and the **Python standard library**. The *reference implementation* is **CPython**. It is kept up-to-date with the latest Python language features. This software can be downloaded from www.python.org.

2. For efficient software development, you need a **professional IDE** (*integrated development environment*) that supports Python programming. Typically, a professional IDE supports code editing with syntax highlighting, intelligent code completion, snippets, code refactoring, build automation, versioning, running and debugging programs. In addition, such an IDE often supports multiple programming languages.

In addition, we can write, execute, and visualise small and simple Python programs on cloud-based environments. For example, there is the "***Try it yourself***" editor on www.w3schools.com/python. The w3schools site has also an online Python tutorial. For example, we can ***visualise*** and debug Python code on pythontutor.com/visualize.html.

Finally, the **CPython** environment includes a simple Python code editor called **IDLE**. IDLE does not support much more than code editing with code highlighting, running, and debugging programs.

[1] NB! Python 3 is not backwards compatible with Python 2. **'Quick Start to Python' focuses exclusively on Python 3**.

A guide to a variety of Python books is available on wiki.python.org/moin/PythonBooks. In addition, the guide also lists Python books in many other languages than English.

The formal Python documentation is available on docs.python.org.

1.3 Scripts and modules, running a Python program

Generally speaking, a *script* is a text file containing a logical sequence of orders capable of being executed by an interpreter without being compiled.

A Python ***script*** is a source text file that is intended to be directly executed by the user. In this context, 'script' is an informal term that means **top-level program file**.

A ***module*** is a Python source text file that is designed to be imported and used from another Python source text file, or from the interactive interpreter.

Python is an interpreted programming language. To execute a Python program, we need a Python interpreter. An interpreter runs through each line of the program and executes the program on the fly.

The Python interpreter is able to run Python code in two different ways:

1. As a script or module
2. As a piece of code typed into an interactive session

Python program running environment / running platform

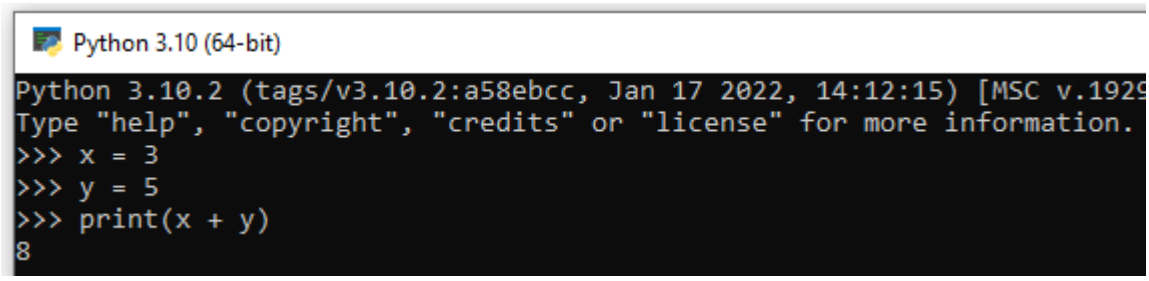
- The Python source code is saved as text file with the ***.py*** extension.
- When the program is to be executed, the Python interpreter loads and interprets the Python source code.
- The Python program is distributed in its human-readable source code.
- Syntax errors in the source code can make the program crash when it is executed.

Python comes with a collection of library modules referred to as ***standard library.***

Executing Python in the interactive mode

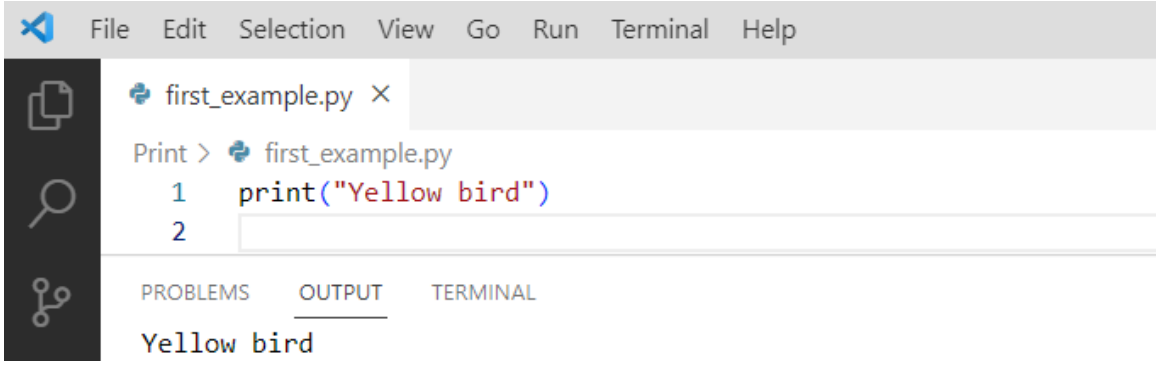
When you install Python on your computer, the default Python interpreter is included. We can start it in the interactive mode by typing ***python*** or ***py*** in the Windows search box or in the command shell. In the interactive mode, we can see **>>>** at the beginning of the line.

In the example below, we have started the Python interpreter in the interactive mode and executed some Python code interactively.



Executing a Python program in an integrated development environment (IDE)

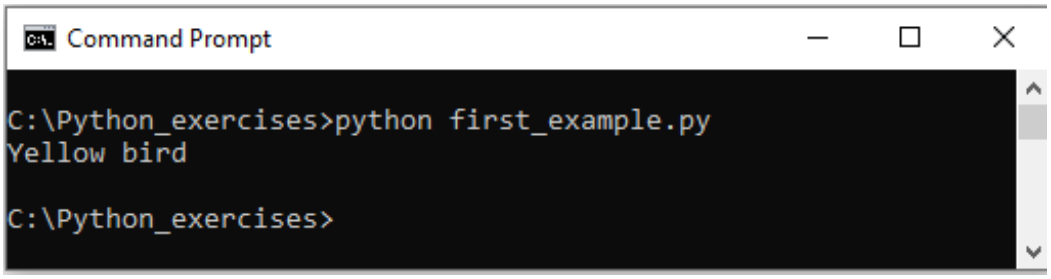
In the example below, we have executed a Python program in the **VS Code** IDE[2].



We have saved the Python source code as a text file with the **.py** extension (*first_example.py*).

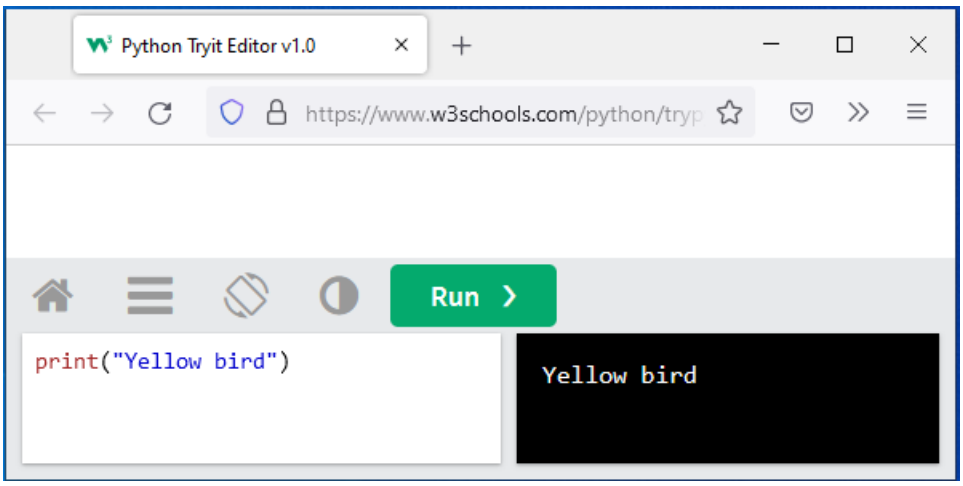
Executing a Python program from command-line

In the example below, we have executed a Python program (*first_example.py*) in the OS console window by invoking the Python interpreter with the name of the program source text from command line.



Executing Python code online

In the example below, we have executed Python code online with w3schools' Tryit Editor (https://www.w3schools.com/python).



Executing server-side Python

There are many Python frameworks for web development in Python (Flask, Django, Bottle, CherryPy, Pyramid, web2py etc.). These are not in the scope of this material.

In addition, we can configure Python scripts to be run as operation system services etc.

[2] In the example, the program is run with the Code Runner extension. Normally, the OS shell also shows some messages.

1.4 Comparing a Python program to a JavaScript program and a Java program

Python source code of a simple script

powers_of_two.py	**Output**

`power = 1`
`while power < 20:`
`  print(power)`
`  power = power * 2`
	1
	2
	4
	8
	16

JavaScript source code of a simple script

powers_of_two.js	**Output**

`var power = 1;`
`while (power < 20) {`
`  console.log(power);`
`  power = power * 2;`
`}`
	1
	2
	4
	8
	16

Java source code of a simple program

PowersOfTwo.java	**Output**

`public class PowersOfTwo {`
`  public static void main(String[] args) {`
`  int power = 1;`
`  while (power < 20) {`
`  System.out.println(power);`
`  power = power * 2;`
`  }`
`  }`
`}`
	1
	2
	4
	8
	16

Similarities in the above three codes

- The while loop
- The assignment symbol (Note: '=' is an *operator* in JS and Java. in Python, '=' is a *delimiter*)
- The operators (comparison, multiplication)
- The actual algorithm which includes four quite similar code lines

How the Python source code differs from the JS and Java codes?

- The file extension is ***.py***
- In this simple Python script, the code is not included in any function. The Python interpreter will start executing the script from the first line of the code.
- The Python code is the shortest one. In Python, there are no braces { } indicating the start and end of the body of the while loop. Instead, Python uses the colon and indentation for this purpose. In addition, there are no semicolons, and the loop condition is not required to be enclosed within parentheses.
- The name of the print function/method is different in each language.
- The variable is defined by writing the name of the variable and assigning a value to the variable[3].

[3] (More advanced explanation) Actually, the value that is assigned to the variable is a reference (~link) to an int object. The variable itself is not bound to any data type. Based on the type of the object whose reference the variable currently holds, Python knows what operations are allowed to be applied on the variable. In Python, arithmetic operations are allowed only on numeric data. If we later assign a string to the variable, then arithmetic operations are no more allowed on the variable.

2 Program structure

2.1 First Python program examples

Our first example has only one line of code. The **print**() function prints output to the console.

hello.py

```
print("Hello!")
```

The **input**() function prints the prompt to the console, takes input from the user at the command line and returns the whole line as a single string when the user presses the *Enter* key. To create a *variable*, we just write a variable name and assign it a value[4].

greeting.py

```
first_name = input("Enter your first name: ")
print("Hello " + first_name)
```

In Python, ***indentation*** is mandatory. You must introduce the indented part with the ***colon*** character.

Always use ***four spaces*** for indentation

The Python code layout details are specified in ***Style Guide for Python Code***

NB! Always use a code formatter that formats your code automatically.

Many programming languages like JavaScript and Java use braces **{ }** to define a block of code. Python uses ***colons*** and ***indentation***. In Python, a code block (loop body, function body etc.) starts with indentation and ends with the first unintended code line.

- In short: ***always*** use ***four spaces*** for indentation.
- Indentation ***must*** be consistent spaces and tabs cannot be mixed.
- The block of code ***must*** be introduced with the ***colon*** character (:) on the code line which precedes the block of code.

check_age.py

```
age = int(input("Enter your age: "))
if age < 18:
    print("You are a minor")
else:
    print("You are an adult")
```

Please notice the following:

- The **input**() function always returns a string. A string and an **int** cannot be compared to each other. Therefore, the return value of the **input**() function should be converted to **int** with the **int**() function.
- Conditions are not required to be enclosed within parentheses in control statements.
- No semicolons!

In the next example, we have an if statement inside a while loop. The script inputs participant ages from the user until the user enters a negative value. Per each iteration the script determines if the participant is a minor or adult.

[4] (More advanced explanation) In Python, all variables refer to objects. In Python, the *value* of a variable cannot be 'undefined' because we cannot create a variable without assigning a value to the variable. In addition, more than one variable can refer to the same object.

check_participant_ages.py

```
age = int(input("Enter the participant's age: "))

while age >= 0:
    if age < 18:
        print("The participant is a minor")
    else:
        print("The participant is an adult")

    age = int(input("Enter the participant's age: "))

print("Bye!")
```

2.2 Python naming convention

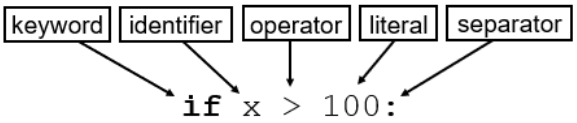
Names of ***variables***, ***functions***, ***methods, scripts,*** *and* ***modules*** should be lowercase with underscores separating words. Names of ***constants***[5] should be uppercase with underscores separating words. Names of ***classes*** should start with an uppercase letter.

Normally, a name can contain ***uppercase and lowercase letters*** A through Z, the ***underscore*** (_) and, except for the first character, the ***digits*** 0 through 9[6].

Names, string literals and keywords are case-sensitive.

2.3 Symbols in a Python source text, whitespace, comments, docstrings

Symbols in a Python source code can be categorised as follows: ***identifiers***, ***keywords, operators, literals,*** and ***separators***



Identifiers are *user-definable names* of variables, functions, methods etc.

Keywords (e.g., `if`, `for`, `while`) are words that are reserved for special purposes in the programming language specification. Keywords cannot be used as *identifiers*.

Operators indicate which operations to apply to their *operands.*

Literals are source code representations of fixed values.

Separators (delimiters) separate some language features from other language features in source code. For example, colon is used to indicate the start of a code block and comma is used to separate items in parameter lists.

White space (*whitespace*) is any non-visible character or series of non-visible characters that represent horizontal or vertical space in typography. The most common whitespace characters may be typed via the *space bar,* the *tab key*, and the *enter key*. Each of these characters has its own character code that is saved in the source text file.

---

[5] In Python, a *constant* is an ordinary variable that assigned a value only once and we can expect that the value does not change. Python has no special syntax for constants. Here is an example of a constant: `PI = 3.1415926535`

[6] In addition to the previously mentioned characters, a name can contain some other characters but normally it is not recommended to use them in the name.

A ***comment*** is a programmer-readable explanation in the source code. Usually, compilers and interpreters ignore comments. Python supports *single-line comments*. Python ignores everything from the hash character (**#**) to the end of the physical line.

```
# This is a comment
```

A ***docstring*** (*documentation string*) is a special multi-line string literal that is meant to be used to document a specific segment of code. A docstring is declared using triple **double quotes**.

```
""" This is a docstring
    with two lines.
"""
```

Python saves all docstrings as metadata that can be accessed with the **help**() function etc. In addition, docstrings are accessed by the *doctest* module that can be used in simple unit testing. We can also assign them to variables for some special purposes.

2.4 Maximum code line length / How to split one line of code over two or more lines

The **Style Guide for Python Code** gives the recommendation to limit all code lines to a maximum of ***79 characters*** to improve readability and keep the quality of the code consistent.

In Python, ***statements cannot cross line boundaries*** except where a line break is allowed by the syntax or *lines are joined based on the line joining rules*.

Code	Line joining rules explained
`x = 10000 + 20000 + 30000` `+ 40000 + 50000`	ERROR! Python requires the whole statement to be on a single ***logical line***.
`x = 10000 + 20000 + 30000 \` `  + 40000 + 50000`	OK! Two or more ***physical lines*** may be joined into a single ***logical line*** using the backslash character ('\')
`x = (10000 + 20000 + 30000` `  + 40000 + 50000)`	OK! Expressions in *parentheses*, *square brackets* or *braces* can be split over more than one physical line without using the backslash character.
`text = "This is a very long` `string literal"`	ERROR! Python requires the whole string to be on a single logical line.
`text = "This is a very long" \` `  "string literal"`	OK! Please notice that on each line the subpart of the string should be enclosed within single or double quotes.
`text = ("This is a very long"` `  "string literal")`	OK! Please notice that on each line the subpart of the string should be enclosed within single or double quotes.

3 Printing to the console, formatting output

The **print**() function prints output to the console. By default, it *moves the cursor to a line* after printing the output. In addition, we can include one or more *newline characters* (**\n**) in the string to be printed to create more empty lines to the output.

Printing each time ***on a separate line***	**Output**

`print("Python 3.10")`
`print()`
`print("Printing in a loop\n")`
 	
`for i in range(3):`
`  print(i)`
	`Python 3.10`
	
	`Printing in a loop`
	
	`0`
	`1`
	`2`

We can print several times on a single line if we set the value of the **end** argument[7]. The print() function appends the value of the end argument to the output instead of a line break.

Printing output ***on a single line*** (preventing line breaks)	**Output**

`power = 1`
`while power < 50:`
`  print(power, end=" ")`
`  power = power * 2`
	`1 2 4 8 16 32`

In Python, we can use **f-strings** (*formatted string literals*)[8] when we want to mix literal text and values of variables and/or any other expressions[9]. We can also assign f-strings to variables and concatenate them with other strings etc. A normal f-string begins with an **f** or **F** before the opening quote and uses braces for placeholders[10].

Using f-strings to format the output	**Output**

`name = "Jennifer"`
`age = 19`
 	
`print(name)`
`print(f"The age is {age} years")`
`print(f"{name}'s age is {age} years")`
	`Jennifer`
	`The age is 19 years`
	`Jennifer's age is 19 years`

`x = 3`
`y = 5`
`print(f"{x} * {y} is {x * y}")`
	`3 * 5 is 15`

Formatting decimal places	**Output**

`price = 17.50`
`pi = 3.1415926535`
 	
`print(f"The price is {price:.2f} euros")`
`print(f"{pi:.4f}")`
	`The price is 17.50 euros`
	`3.1416`

[7] The end parameter is a *keyword argument* (named argument) which includes the parameter name, '=', and the value. Although keyword arguments are not positional, they cannot precede any positional argument in the function call.
[8] F-strings were introduced in Python 3.6. They are not available in older versions of Python.
[9] We can write the f-string directly to the print function's parameter list, or first assign it to a variable and then write the name of the variable to the print function's parameter list.
[10] An f-string can also begin with 'fr' or 'rf' (upper or lowercase) before the opening quote if we want it to be treated as a raw string. A raw string cannot contain an escape sequence. Therefore, the backslash is treated as a normal character in a raw string.

Formatting leading zeros	Output

`hours = 9`
`minutes = 5`
`print(f"{hours}:{minutes:02d}")`
`print(f"{hours:02d}:{minutes:02d}")`
	
	`9:05`
	`09:05`

In a placeholder, we can add several different kinds of format specifiers after the colon character.

Placeholder in a f-string / Some of the format specifiers	
{	Opening brace
expression	The variable or some other type of an expression to be formatted
:	Colon
fill	If a valid *align* value is specified, it can be preceded by a **fill character** that can be any character. The default fill character is space.
align	**<** forces the field to be **left-aligned** within available space (default for most types) **>** forces the field to be **right-aligned** within available space (default for numbers) **^** forces the field to be **centred** within available space
sign	**+** indicates that a **sign should always be used** Space indicates that a **leading space** should be used for positive numbers
#	**#** causes a type-specific alternate form to be used (valid for numbers only)
0	When no explicit alignment is given, preceding the width field by zero enables sign-aware **zero-padding for numeric types**.
width	Defines the ***minimum*** total field width
grouping	**_** indicates the use of an **underscore for a thousand separator** **,** indicates the use of a **comma for a thousand separator**
. *precision*	For float and Decimal types, the field indicates the **number of decimal places** For string presentation types the field indicates the **maximum field width** This field is not allowed for integer types.
type	String presentation type **s** **String** format. This is the default and *may be omitted*. Integer presentation types **d** **Integer** format (base 10). This is the default and *may be omitted*. **b** **Binary** format (base 2). Outputs the number as a ***binary number***. Presentation type for **float** and **Decimal** **f** Fixed-point notation. For a given precision p, formats the number as a decimal number with exactly p digits following the decimal point.
}	Closing brace

We can use the **width** specifier when we want to align output into columns.

Aligning output into columns	Output

`p1 = "Jennifer"`
`p2 = "Joe"`
`p1_salary = 1200.00`
`p2_salary = 952.75`
 	
`print(f"{'Name':10s}{'Salary':>8s}")`
`print(f"{p1:10s}{p1_salary:8.2f}")`
`print(f"{p2:10s}{p2_salary:8.2f}")`
	`Name  Salary`
	`Jennifer  1200.00`
	`Joe  952.72`

`print(f"{'Number':6s}{'Square':>8s}{'Cube':>8s}")`
`for x in range(2, 5):`
`  print(f"{x:6d}{x*x:8d}{x*x*x:8d}")`
	`Number  Square  Cube`
	`  2  4  8`
	`  3  9  27`
	`  4  16  64`

Here is how we can format dates and times with an f-string[11].

Formatting date and time	Output

`from datetime import datetime`
`dt = datetime.today()`
`print(f"{dt.day}.{dt.month}.{dt.year}")`
`print(f"{dt.hour}:{dt.minute:02d}:{dt.second:02d}")`
	`30.5.2022`
	`9:05:10`

Lists and other iterables can be printed without looping.

Printing a list	Output

`number_list = [5, 3, 7, 1]`
`print(number_list)`
`print(*number_list)`
`print(*number_list, sep=", ")`
`print(*number_list, sep=" | ")`
`print(*number_list, sep="")`
`print(*number_list, sep="\n")`
	`[5, 3, 7, 1]`
	`5 3 7 1`
	`5, 3, 7, 1`
	`5 | 3 | 7 | 1`
	`5371`
	`5`
	`3`
	`7`
	`1`

There are several different ways to build a string to be printed.

Three different ways to build a string to be printed	Output
`name = "Jennifer"` `age = 19`	
`print(f"{name} is {age} years of age")`	`Jennifer is 19 years of age`
`print(name, "is", age, "years of age")`	`Jennifer is 19 years of age`
`print(name + " is " + str(age) + " years of age")`	`Jennifer is 19 years of age`

Note 1: Using **f-strings** is the recommended way!
Note 2: In Python, only strings can be joined with the built-in '+' operator.

[11] We can also format dates and times with the strftime() method.

4 Inputting data from the user

4.1 Inputting data

The **input**() function prints the prompt to the console, takes input from the user at the command line, and returns the whole line as a single string when the user presses the *Enter* key.

input_example_1.py

```
first_name = input("Enter your name: ")

print(f"Hello {first_name}!")
```

The **input**() function always returns a single string. If the program wants a numeric type, then we need to convert the string to **int** with the **int**() function, or to **float** with the **float**() function.

input_example_2.py

```
quantity = int(input("Enter the quantity: "))
price = float(input("Enter the price: "))
total_price = quantity * price

print(f"The total price is {total_price:.2f}€")
```

4.2 Handling invalid input | Exception handling

Exceptions make it possible to break out of the normal flow of control of a code block in order to handle an error or another exceptional condition. Python raises an exception whenever it detects an error in a program at runtime. If we ignore the error, Python terminates the program and prints an error message.

To avoid the program to crash, we can use the ***try statement*** to catch and recover from the exception. The try statement must include at least one ***error handing clause***. If a specific type of an error occurs within a try clause, the program execution jumps to the corresponding error handling clause, and the program can resume execution after the try statement.

The program below crashes if the conversion from string to int fails[12].

print_cube.py

```
x = int(input("Enter an integer: "))

print(f"The cube of {x} is {x ** 3}")
print("Bye!")
```

For example, if the user enters '`five`', the **int**() function notices that the string cannot be converted to int, and the **int**() function raises an exception of type **ValueError**.

Raising an exception terminates the execution of the function. If the program ignores the exception, it crashes, and Python prints an error message as below.

```
Enter an integer: five
Traceback (most recent call last):
  File "c:\examples\print_cube.py", line 1, in <module>
    x = int(input("Enter an integer: "))
ValueError: invalid literal for int() with base 10: 'five'
```

[12] Note: To check if the inputted value is numeric, we cannot use the *type()* function because the **input**() function always returns a ***string***. Neither can we check if the inputted value is numeric using the string handling methods *isdigit()*, *isdecimal()*, or *isnumeric()* because they do not accept the sign (**-** **+**) and the decimal separator (**.**).

This can be avoided by using the **exception handling** mechanism as below.

exception_handling_1.py

```
try:
    x = int(input("Enter an integer: "))
    print(f"The cube of {x} is {x ** 3}")
except ValueError:
    print("The entered value is not an integer")

print("Bye!")
```

If the conversion from string to int in the **try** clause fails, then (1) the execution of the code in the **try** clause ends and (2) the program execution jumps to the **except** clause. (3) After the code in the **except** clause is finished, the program execution moves out from the try-except structure and continues normally. If the conversion from string to int succeeds, Python skips the **except** clause and the program execution moves out from the try-except structure and continues normally.

We can also use the **else** clause as shown in the example below. If no exception is raised in the **try** clause, then Python executes the **else** clause.

exception_handling_2.py

```
try:
    x = int(input("Enter an integer: "))
except ValueError:
    print("The entered value is not an integer")
else:
    print(f"The cube of {x} is {x ** 3}")

print("Bye!")
```

If we want to ***print the inputted string after the conversion to numeric fails***, then we should assign the string to a variable before trying to convert it to numeric.

exception_handling_3.py

```
try:
    x = input("Enter an integer: ")
    x = int(x)
    print(f"The cube of {x} is {x ** 3}")
except ValueError:
    print(f"{x} is not an integer")

print("Bye!")
```

A **try** statement can include *several **except** clauses* and one **finally** clause. Python executes the **finally** clause either after the **try** clause or after an **except** clause if an exception is raised.

exception_handling_4.py (*Advanced example*)

```
try:
    divider = int(input("Enter divider: "))
    print(f"10 / {divider} = {10 / divider}")
except ValueError:
    print("The entered value is not an integer")
except ZeroDivisionError:
    print("Division by zero is not allowed")
finally:
    print("Bye!")
```

5 Variables and data types

5.1 Overview

In Python, we define a variable by writing the name of the variable and assigning a value to the variable.

```
count = 0                    # int
population = 1_380_004_385   # int    Underscores can be used to improve readability
price = 19.35                # float
total = 10_500_000_025.42    # float  Underscores can be used to improve readability
city = "Helsinki"            # str    String literals can be enclosed in double quotes
country = 'Finland'          # str    String literals can be enclosed in single quotes
is_found = False             # bool   The bool literals are True and False
```

Python is a ***dynamically typed*** language[13]. Python performs type checking at run-time as opposed to statically typed languages (such as Java) that perform type checking at compile-time. That is, Python does not know about the type of the variable until the code is run.

Python binds the variable to a type when it executes the operation which assigns a value to the variable. The type of a variable determines the range of values the variable can have and the set of operations that are available for the type. In addition, the type of a variable can change during the program execution.

```
x = 6          # The first assignment defines x as a variable of type int
x = 1.25       # After the second assignment, x is of type float
x = "Hello!"   # After the third assignment, x is of type str
```

Below is a list of selected data types in Python.

Data type category		Type in Python	Explanation
Numeric data	*Integer*	`int`	Integer of unlimited magnitude
	Decimal number	`float`	The precision is machine-dependent. Generally: 64 bit double-precision IEEE 754 *floating point* 15 digits of precision, not always accurate
	Complex number	`complex`	Complex number with real and imagery parts[14]
Truth value	*Boolean*	`bool`	Allowed values: `True`, `False`
Textual data	*String* ** Sequence*	`str`	A string of **0 – n** Unicode (UTF-16) characters
	** Sequence*	`range`	Sequence of numbers commonly used in for loops
Collection	** Sequence*	`list`	Ordered collection. Numeric indexing.
		`tuple`	Immutable ordered collection. Numeric indexing.
	Set	`set`	Unordered collection that does not allow duplicates
		`frozenset`	Immutable set
	Dictionary	`dict`	Collection of key-value pairs. Indexed by key.
		`None`	Has one value: `None` which represents *null*.

[13] (More advanced explanation) A binding is ***static*** if it first occurs before run-time and remains unchanged throughout program execution. A binding is ***dynamic*** if it first occurs during execution and/or it can change during execution of the program. In a statically bound language, the declaration of a variable specifies the variable's permanent type. That is, the language knows about the type of the variable when the code is compiled.

The ***advantages of static binding*** include early detection of type errors, type safety, and lower cost in performance. The disadvantages include less flexibility and often more boilerplate code. The ***advantages of dynamic binding*** include more flexibility and less boilerplate code. The disadvantages include late detection of type errors, lower type safety, and higher cost in performance.

[14] Complex numbers are out of scope of this material.

Python is also ***strictly typed*** language. For example, there are strict rules for the allowed types of operands in expressions. For example, we are allowed to perform arithmetic operations on numeric values only, and both operands of the string concatenation operator (+) must be of type **str**.

```
print(1 + 2)            # 3
print(1 + 2.5)          # 3.5          Python promotes the int to float
print("1" + 2)          # ERROR!       Not allowed
print("1" + str(2))     # 12
```

5.2 String type (str), string literals

In Python, a string contains an immutable[15] sequence of Unicode (UTF-16) characters. A string can contain any number of any characters[16]. Practically speaking, a **str** is an immutable array of characters where each character is itself a **str** object of length one.

A string literal is a sequence of characters that Python treats as a single item. There is no difference between strings defined with ***single quotes*** (') and ***double quotes*** (")[17]. If we want to include quote in the string, then we can use single quotes within double quotes and vice versa. Alternatively, we can insert the quote in an ***escape sequence*** by prefixing it by a ***backslash***.

```
print('He said, "Where do we meet?"')   # He said, "Where do we meet?"
print('I said, "At Joe\'s garage."')    # I said, "At Joe's garage."
print("He said, \"Where?\"")            # He said, "Where?"
print("I said, \"At Joe's garage.\"")   # I said, "At Joe's garage."
```

We can use the *newline sequence* ('**\n**') to insert line breaks into the output.

```
print("This\nprints on\nthree separate lines")
```

If we want to include the ***backslash character*** in the string, then we should prefix it by another backslash or write an '**r**' (upper or lowercase) before the opening quote to specify a ***raw string***. A raw string cannot contain an escape sequence. Therefore, Python treats the backslash character as a normal character in a raw string.

```
print("C:\\Python310\\json")            # C:\Python310\json
print(r"C:\Python310\json"  )           # C:\Python310\json
```

Here are some common ***escape sequences***:

\"	Double quote
\'	Single quote
****	Backslash
\n	Newline
\o*ov*	Octal value *ov*
\x*hv*	Hex value *hv*
\u*xxxx*	16-bit Unicode character with hex value *xxxx* See https://www.unicode.org/charts
\N{*name*}	Character named *name* in the Unicode database

```
print("\u2660 \u2663 \u2665 \u2666")    # ♠ ♣ ♥ ♦
print("\N{BLACK SPADE SUIT}")           # ♠
print("\N{EURO SIGN}")                  # €
```

[15] That is, the content of the string cannot be modified once it is created. For example, `given_name.upper()` constructs and returns a new string instead of changing the existing content.
[16] Letters, digits, and special characters.
[17] In addition, ***docstrings*** are written in ***triple double quotes*** (or triple single quotes)

5.3 Boolean type (bool), truthy and falsy values

A **bool** value represents a truth value (true / false). Python uses the keywords **True** and **False** to represent truth values. In Python, any object can be tested for truth value. A *falsy* value is a value that is considered false. A *truthy* value is a value that is considered true.

Falsy values; Python considers the following false:

- **`False`**
- Zero of any numeric type
- An empty sequence or collection: `""`, `''`, `()`, `[]`, `{}`, `set()`, `range(0)`
- **`None`**

Truthy values; Python considers the following true:

- **`True`**
- A non-zero value of any numeric type
- A non-empty sequence or collection.

In an arithmetic operation, Python converts **`False`** to zero and **`True`** to one.

bool_example.py

```
if 2 * 2 == 4:
    print(True)       # True
if True:
    print(True)       # True
if 123:
    print(True)       # True
if "Hello!":
    print(True)       # True
if [1, 2, 3]:
    print(True)       # True

print(1 > 2)          # False
print(bool(None))     # False
print(bool(""))       # False
print(bool([]))       # False

print(5 + True)       # 6
print(3 * False)      # 0
```

5.4 Explicit type conversions

We can perform some explicit type conversions with Python's built-in functions.

Function	Result
`int``(x)`	x converted to int
`float``(x)`	x converted to float
`str``(x)`	x converted to str
`bool``(x)`	x converted to bool
`chr(x)`	String representation of integer x. E.g. `chr(65)` returns `"A"`
`ord(x)`	Integer representation of character x. E.g. `ord("A")` returns 65
`bin(x)`	Binary string representation of integer x. E.g., bin(14) returns "0b1110" `print(f"{14:b}")` prints `1110`

5.5 Scopes of variables

The ***scope** of a variable* is the part of the code where the variable is visible. The ***lifetime** of a variable* is the time during which the variable stays in memory.

Here is a simplified summary of scopes of local and global variables ***in Python***[18].

	Local variable	Global variable
Creation	Created in a function, either in the parameter list of the function or in the body of the function.	Created at the main level of a Python code file. ... or created in a function with the help of the *global* keyword...
Scope	**Local scope** The scope starts from the variable definition and continues to the last line of the function body. The variable can be visible only within the function.	**Global scope** After the variable is created, the scope covers the whole code file. The variable can be visible within the whole code file.
Lifetime	Within the function only	Within the whole program

The example below demonstrates the ***local scope*** of variables.

Scope example 1	Comments

`def fun(count):`
`  x = 21`
`  print(f"x is {x}")`
`  print(count)`
 	
`def main():`
`  x = 1.25`
`  fun(42)`
`  print(f"x is {x}")`
 	
`main()`
	The **fun**() function has two **local variables**: **`count`** and **x**
	Note: x must be defined before it can be used in this function.
	Prints **`x is 21`**
	Prints **42**
	
	The **main**() function has one **local variable**: **x**
	Note: x must be defined before it can be used in this function.
	We call the fun function with **42** as the argument
	Prints **`x is 1.25`**

The example below demonstrates the ***global scope*** of variables.

Scope example 2	Comments

`def bad():`
`  print(x)`
 	
`bad()`
	The program ***fails*** here because x is not defined anywhere.

`def bad():`
`  print(x)`
 	
`x = 123`
`bad()`
	Prints **123**
	
	This defines **x** as a ***global*** variable with the initial value of **123**.
	NB! **x** must be defined *before* we call the bad function!

[18] In addition, there is the 'nonlocal' scope when we create inner functions, and the built-in scope which includes Python's built-in variable names.

By default, the first assignment to a variable within a function defines a new local variable. We can change this by using the keyword **`global`** to tell Python that a certain name is a global name.

Scope example 3	Comments

`def bad():`
`  x = 42`
`  print(x)`
 	
`x = 21`
`bad()`
`print(x)`
	Defines **x** as a ***local variable*** with the initial value of **42**
	Prints **42**, the value of the ***local variable*** named x
	
	Defines **x** as a ***global*** variable with the initial value of **21**
	
	Prints **21**, the value of the ***global variable*** named x

`def bad():`
`  global x`
`  x = 42`
`  print(x)`
 	
`x = 21`
`bad()`
`print(x)`
	The keyword **`global`** specifies that the scope of **x** is ***global***
	Assigns **42** to a ***global variable***
	Prints **42**, the value of the ***global variable*** named x
	
	Defines **x** as a ***global*** variable with the initial value of **21**
	
	Prints **42**, the value of the ***global variable*** named x

Best practice: Avoid using global variables.

Best practice: We should ***avoid using global variables*** as much as possible. They should **NOT** be used to bypass function parameters and return values. If many functions are accessing the same global variables in a large software project, then it becomes extremely difficult to track the flow of data in a robust way. In this type of situation *unit testing* and *code refactoring* becomes very difficult.

BAD_code.py

```
def cube(a):
    global x
    x = a ** 3

cube(5)
print(f"Cube is {x}")
```

Passing data into and out of functions should be handled by using arguments and return values.

good_code.py

```
def cube(a):
    return a ** 3

def main():
    x = cube(5)
    print(f"Cube is {x}")

main()
```

6 Basic operators and expressions, mathematical calculations

6.1 Operators and expressions

An ***expression*** is something that can be evaluated to a value of a certain data type. Expressions are constructed from ***operators*** and ***operands***. The operators of an expression indicate which operations to apply to the operands. Examples for operands are literals, variables, function calls, and method calls.

Usually, *almost any expression of type **T*** can be used where a single value of type ***T*** can be used. For example, if the syntax requires us to write an *int* in our code, we can usually write there almost *any expression that evaluates to an int*.

When an expression contains multiple operators, the ***precedence*** of the operators controls the *order in which the individual operators are evaluated*. For example, Python evaluates the expression **1 + 2 * 3** as **1 + (2 * 3)** because the multiplication operator has higher precedence than the addition operator. The evaluation order can be changed by using parentheses.

In the case of same level of the precedence, ***associativity*** *rules* define the order of evaluation (from ***left to right*** or ***right to left***). In Python, most of the operators have *left associativity*. For example, in the expression `x > y == True` both operators are on the same level of the precedence. *Left associativity* means that the expression should be evaluated as `(x > y) == True`

The below shows some Python operators[19] in decreasing order of precedence from top to bottom.

Operators		Comments	Associativity
Parenthesised expr. Collection display	`()` `[ ]  { }`	Parenthesised expression or binding List display, Set or Dictionary display	left to right

Indexing
Slicing
Function call
Attribute reference
	`[i]`
	`[a:b]`
	`x()`
	`x.`*attribute*
			left to right

Exponentiation	`**`	`5 ** 3` `2 ** (x + y)`	***right to left***
Unary minus	`-`	`-(2 + 3) + 4`	***right to left***

Multiplication
Division
Floor division
 	
Remainder
	`*`
	`/`
	`//`
	
	`%`
		
		Division yields a **float**
		Floor division rounds down to nearest integer, its result type is either a float or an int.
		% yields the remainder of *floor division*.
			left to right
			left to right
			left to right
			
			left to right

Addition, Subtraction String concatenation	`+  -` `+`	The meaning of `'+'` depends on its operands	left to right left to right

Comparison
 	
Membership test
Identity test
	`<  <=  >  >=`
	`==  !=`
	
	`in  not in`
	`is  not`
		NB! All comparison operators are at the same level.
		Equal to (*value equivalence test),* not equal to
		Python supports chained comparisons:
		17 < age < 65 means 17 < age **and** age < 65
			Not applicable for chained comparisons
			
			left to right
			left to right

Not	`not`		left to right
And	`and`	Note: *short-circuit evaluation*[20]	left to right
Or	`or`	Note: *short-circuit evaluation*	left to right
Conditional expression	`if – else`	`x = 0 if x > 10 else x + 1`	left to right
Lambda	`lambda`	`lambda a : int(a) + 1`	left to right
Assignment expression	`:=`	To be used as a part of a larger expression	***right to left***

[19] (Advanced explanation) In addition, Python supports operator overloading.

[20] Python "***short-circuits***" the evaluation of the logical operators. If the left-hand side condition of the ***not*** operator is ***True***, then the right-hand side condition of the ***not*** operator is not evaluated. If the left-hand side condition of the ***and*** operator is **False**, then the right-hand side condition of the ***and*** operator is not evaluated.

Some of the above symbols (+, *, **, if, else) have *context-sensitive semantics*. For example, depending on the context, the star (*****) can mean arithmetic multiplication, creating multiple copies of a sequence, unpacking an iterable object, variable number of arguments, or 'all content' when importing a library module.

Python understands '**+**' as string concatenation operator if at least one of its operands is a string. In Python, string concatenation is only possible between two strings.

Example	Result	Comment
`"cat" + "fish"`	`"catfish"`	Concatenate two strings
`"12" + str(34.56)`	`"1234.56"`	Concatenate a string and a value of another type

Assignments

In Python, assignment is a *statement*. That is, we cannot use an assignment as an expression or a part of an expression. The below shows some forms of the assignment statement.

Simple assignments	
Basic assignment statement	`age = 21`
Assign different values to different variables	`x, y = 4, 2`
Assign the same value to multiple variables	`i = j = k = 0`
Augmented assignments	

Augmented assignment is the combination of a binary operation and an assignment statement	`x += 1 # x = x + 1`
	`x -= 1 # x = x - 1`
	`x *= 5 # x = x * 5`
	`x //= 2 # x = x // 2`
	`x /= 2 # x = x / 2`
	`x %= 2 # x = x % 2`

Python evaluates the right-hand side of an assignment before the left-hand side.
Note: Python does not have the **++** and **--** operators.

6.2 Functions for mathematical computations

There are several built-in functions for mathematical operations. Here are some of them.

Built-in functions	
abs(*x*)	Absolute value of x
divmod(*x, y*)	Computes x // y and x % y and returns the quotient and remainder
round(x, *n*)	Rounds a number to *n* digits, rounding half to even. If *n* is omitted, it defaults to 0
pow(*x, y*) **pow**(*x, y, z*)	x ** y x ** y % z

The Python standard library includes several modules for mathematical computations. Below are some examples of functions in the **math** and **statistics** modules.

Functions in the math module	
math.**sqrt**(*value*)	Returns the square root of the parameter
math.**ceil**(*value*)	Rounds a number up to the nearest int
math.**floor**(*value*)	Rounds a number down to the nearest int
Functions in the statistics module	
statistics.**mean**(*iterable*)	Arithmetic mean (average) of data
statistics.**median**(*iterable*)	Median (middle value) of data

Examples

builtin_math_functions.py

```
print(round(6.49, 1))          # 6.5  (float)
print(round(6.49))             # 6    (int)
```

math_module_functions.py

```
import math

print(math.sqrt(36))           # 6.0  (float)
print(math.ceil(1.01))         # 2    (int)
print(math.floor(5.99))        # 5    (int)
```

statistics_module_functions.py

```
import statistics

data = [1, 2, 3, 7, 9]
print(statistics.mean(data))    # 4.4
print(statistics.median(data))  # 3
```

In addition, the **max**() and **min**() functions can take more than two parameters.

max_and_min.py

```
x = 1
y = 7
z = 5
print(max(x, y, z))            # 7
print(max(x, y, z, 8, 9))      # 9
print(min(x, y, z))            # 1
```

NB! There are many commonly used third party Python libraries for statistics and data science, like **NumPy** (https://numpy.org) and **Pandas** (https://pandas.pydata.org). For example, the Python Package Index web site (https://pypi.org) is a useful repository of these.

6.3 Type conversions

Calculations and type conversions

Python is a ***strongly typed*** programming language. For all operations, Python requires type-compatible operands. Some binary operations on values of different types are allowed. In arithmetic operations, we can mix ints and floats. In such situations, Python *implicitly* converts the ints to floats to be able to perform the binary operation. In addition, we can do ***explicit type conversions*** with the **int**(), **float**(), **str**(), and **bool**() functions. A type conversion always results to a new value.

Example	Result	Type of the result	Comment
`5 + 4.25`	`9.25`	`float`	First, Python ***promotes*** int to float, then addition
`1 / 2.0`	`0.5`	`float`	First, Python ***promotes*** int to float, then division
`1.0 / 2.0`	`0.5`	`float`	Division (*no type conversion)*
`1 / 2`	`0.5`	**`float`**	First, Python ***promotes*** both ints to float, then division
`1 // 2`	**`0`**	**`int`**	Floor division (*no type conversion)*

Computer arithmetic differs from school arithmetic

Division (/) operator yields the quotient as a float.

```
print(6 / 5)          # 1.2
print(6 / 12)         # 0.5
```

Floor division (//) operator rounds down to nearest integer. Its result type depends on its operands. If both of them are of type int, then the result is of type int. If either operand is of type float, then the result is of type float.

```
print(1 // 2)         # 0
print(6 // 5)         # 1
print(6 // 1.5)       # 4.0
print(8.2 // 2.1)     # 3.0
```

The remainder operator (%) yields the remainder of **floor division**.

```
print(5 % 3)          # 2
print(5.0 % 3)        # 2.0
print(7.0 % 1.25)     # 0.75
```

In arithmetic operations, bool values **False** and **True** behave like the values 0 and 1 .

```
print(3 + False)      # 3
print(3 + True)       # 4
print(3.5 + True)     # 4.5
print(3.5 + (8 > 6))  # 4.5
```

6.4 Inaccurate floating-point values, using Decimal for precise decimal values

Digital computers *cannot represent all decimal numbers exactly*. Python uses a subset of the *IEEE 754 binary floating-point standard* to represent floating point numbers. Floating-point decimal types and floating-point arithmetic are **not 100 % accurate** since some decimal numbers require an infinite number of digits to be represented (e.g., 1.0 / 3.0) and ***some decimal numbers are not exactly representable*** for other reasons (e.g., 0.1). The use of floating-point arithmetic is a trade-off between range/speed and precision/accuracy.

Some floating-point computations can lead to ***inconsistent results***[21] and cause ***potential faults in the program logic***.

```
print(0.15 + 0.15)      # 0.3
print(0.1 + 0.2)        # 0.30000000000000004
```

Here are some examples of different ways to avoid the above-mentioned inconsistency.

1. If the programming language provides a *built-in special decimal data type for accurate decimal numbers* (for example, C# contains the **decimal** data type), then we can use it instead of the built-in floating-point data types.
2. If the programming language (e.g., JavaScript) does not provide any support for accurate decimal numbers, then
   a. We can use an integer data type and do monetary calculations in cents. That is, we save 12.45 as 1245. This way, calculations run fast, but it takes more work to design, write, and test the program code.
   b. We try to minimise the issue by using separate rounding operations

[21] This cannot be accepted in monetary calculations etc.

c. Alternatively, we can use some third-party library that provides the required functionality. Usually, it is a little bit tricky to work with this kind of utilities.

3. If the programming language's own standard library includes a class for manipulating high-precision decimal values, then we can use it.

Using Decimal for precise decimal values

Decimal can deal with theoretically infinitely large and infinitely precise decimal values

The ***decimal*** *module* contains the ***Decimal*** class for manipulating arbitrary-size and arbitrary-precision decimal values. The `Decimal`[22] class provides high-precision arithmetic needed in monetary calculations etc. To perform accurate calculations, we can create objects from the Decimal class as below. The example below shows only some very basic operations.

using_decimal.py

```
from decimal import Decimal

x = Decimal("0.1")
y = Decimal("0.2")

sum = x + y

print(0.1 + 0.2)            # 0.30000000000000004
print(sum)                  # 0.3

print(x - y)                # -0.1
print(x * y)                # 0.02

if sum == Decimal("0.3"):   # NB! Both operands should be of the type Decimal
    print(f"sum is {sum}")  # sum is 0.3

# To input an accurate decimal value
price = Decimal(input("Enter price: "))
```

[22] See https:#docs.python.org/3/library/decimal.html

7 **Selection** (if statement)

The **if** statement executes or skips other statements on the value of a specified condition (*logical expression*). A ***logical expression*** is a *Boolean expression* that results in **True** or **False**. The **if** statement always starts with the keyword if. The else clause and elif clauses are optional. If the else clause is present, it must be specified last.

```
if logical_expression:
    stmt_block

if logical_expression:
    stmt_block_1
else:
    stmt_block_2
```

```
if logical_expression_1:
    stmt_block_1
elif logical_expression_2:
    stmt_block_2
elif logical_expression_3:
    stmt_block_3
else:
    stmt_block_3
```

The if statement syntax requires a single statement or a statement block after the colon.

Example 1. Simple if statement

```
if grade == 0:
    text = "Retake"
```

Example 2. Simple if-else

```
if age < 18:
    text = "Minor"
else:
    text = "Adult"
```

Example 3. Multiple choice (if - elif - else)

```
if age < 0:
    text = "The value is too low"
elif age < 18:
    text = "Minor"
elif age < 65:
    text = "Adult"
elif age < 123:
    text = "Senior"
else:
    text = "The value is too high"
```

Some idiomatic conditions in Python

Chaining comparisons

```
if 0 <= grade <= 5:
    print("Ok")
```

Writing a single test for several values

```
if x in (1, 28, 42):
    print("Ok")
```

Conditional expression

The ***conditional expression*** is not a control structure that directs the flow of program execution. Using the conditional expression, we can write a simple conditional on a single line. It uses three operands: a logical expression (the condition) and two other expressions. The condition is evaluated first. Conditional expressions use *short-circuit* evaluation.

```
first_expression if logical_expression else second_expression
```

If the condition is true, then the result is the value of the first expression. Otherwise, the result is the value of the second expression.

```
larger = a if a > b else b

text = ("The value is too low" if age < 0 else
        "Minor" if age < 18 else
        "Adult" if age < 65 else
        "Senior" if age < 123 else
        "The value is too high")
```

8 Repetition (while loop, for loop)

8.1 While loop

If the repetition control is based on *general condition*[23] rather than a counting the number of repetitions, we normally write a **while loop**[24]. We can use the following structure to avoid potential problems in the program logic:

Python code	Pseudocode

`sum = 0`
 	
`x = int(input("Enter the first value: "))`
`while x > 0:`
`  sum = sum + x`
`  x = int(input("Enter the next value: "))`
 	
`print(f"The sum is {sum}")`
	get the first item
	while (*an item is available*) {
	  process the item
	  get the next item
	}

The above is the standard way to write a ***clean while loop***. The disadvantage is that we have to write twice the statement(s) that get the values. But this is the generic solution that works in all situations. We should not write any adventurous code that complicates the logic or is based on side-effects[25].

Inputting several values per each repetition in a sentinel-controlled loop

Sometimes a special value called a ***sentinel*** *value* is used to indicate end of data entry. In the example below, the program inputs data (*multiplier* and *multiplicand*) from the user until the user enters a sentinel value (*zero*) as the multiplier to indicate that no more values will be entered.

sentinel_controlled_loop.py

```
x = int(input("Enter multiplier (zero ends): "))
while x > 0:
    y = int(input("Enter multiplicand: "))
    print(f"{x} * {y} = {x * y}")

    x = int(input("Enter multiplier (zero ends): "))
```

8.2 For loop

If the repetition control is based on a *certain number of repetitions* rather than a general condition, we normally write a **for loop** instead of a while loop. Examples of this are repeating the loop body five times and iterating a collection of items.

The ***for statement*** in Python differs from what you may be used to in JavaScript or Java. Python's for statement does ***not*** use a counter variable and a condition. Instead, *it* ***iterates over the items of any sequence or iterable collection*** (range, string, list, set, dictionary etc.).

[23] In a case of a generic condition, the number of repetitions cannot be known until the loop begins executing.
[24] Most often, we use the `while` statement instead of a `do-while` statement to keep the code clean.
[25] Please notice that in normal situations it is ***not*** good programming practice to use the `break` statement in the loop body. An equivalent loop can always be written without a `break` statement.

In the first example, the loop body is executed 5 times.

for_example_1.py

```
for i in range(5):
    print("Hello!", end=" ")      # Hello! Hello! Hello! Hello! Hello!
```

The built-in **range**() function generates an arithmetic progression. In the example above, the generated sequence is as follows: `0 1 2 3 4`

The given end point is *never* included in the sequence. The default start point is zero and the default increment is one. It is also possible to set the start point explicitly, and to specify a different increment. The increment can also be negative.

for_example_2.py

```
for i in range(4):
    print(i)                  #  0 1 2 3

for k in range(2, 11, 2):     #  2 4 6 8 10
    print(k)

for x in range(5, 0, -1):     #  5 4 3 2 1
    print(x)
```

The ***for statement*** provides a simple way to iterate a collection in first-to-last order without an index. To iterate over the indexes of a sequence, we can use the **range**() and **len**() functions.

for_example_3.py

```
cities = ["London", "Paris", "Oslo"]

for city in cities:
    print(city, end=" ")                        # London Paris Oslo
```

```
zoo = ["tiger", "lion", "zebra"]

for index in range(len(zoo)):
    print(f"{index}:{zoo[index]}", end=" ")     # 0:Tiger 1:Lion 2:Zebra
```

Python assigns successive values from the collection to the variable on the left of the keyword 'in'. For each value, the body of the loop is repeated.

Note: In Python, there are also shorter ways to print collections. Some of them are shown in chapter 3.

Nested loops

If the body of a loop contains another loop, we have a *nested loop*[26]. For each iteration of the outer loop, the inner loop executes completely. To keep the code clean, we should avoid too many levels of nesting.

nested_loops.py

```
for x in range(1, 6):
    for y in range(1, 6):
        print(f"{x} * {y} = {x * y}")
    print()
```

[26] There is no practical limit for levels of nesting loops and other control statements.

Exiting a loop early with the break statement

Sometimes, we want to continue looping only until a certain condition is met. We can use the **break** statement to exit a loop early. In the following example, we want to print positive integers from the start of the list and exit the loop early when there is a non-positive integer.

using_break.py

```
for x in [5, 6, -1, 2]:
    if x <= 0:
        break
    print(x, end=" ")            # The program prints 5 6
```

9 Defining functions

9.1 Function definitions

To say it in short: A function is a block of code which only runs when it is called.

A ***function definition***[27] defines a *user-defined function* object. Although function definition is an executable statement, it does ***NOT*** execute the function itself. The function gets executed only when it is called. The simplified structure of the function definition is the following:

```
def function_name(parameter_list):
    function_body
```

The first line of the function definition is called ***function header***. It must start with the **def** keyword (abbreviation of *define*) and end with a colon. The name of the function and parentheses follow the **def** keyword. Each statement in the block below the function header must be indented by the same number of spaces. This block is called ***function body***. Python executes the code within the function body only when the function is called.

In the example below, we ***define*** a function named *greet_me* and ***call*** *it* once. The function is as simple as it can be. It does not take any arguments (its parameter list is empty) and there is no return statement in the function.

simple_function_example.py

```
def greet_me():
    print("Hello!")

greet_me()
```

We can call a function by writing its name followed by parentheses[28]. Python reads the program source text **forward only** when it loads it into the memory. That is, a function call cannot be executed successfully if the function to be called is not yet defined. Therefore, the following fails:

```
greet_me()            # NameError: name 'greet_me' is not defined.

def greet_me():
    print("Hello!")
```

Best practice
We include our own **main**() function if we define any other function in our script.

In this course material, we include our own user-defined **main**() function in all those main level program source texts where we define one or more functions. This *best practice* avoids unnecessary global variables, makes it easier to import the module into another program, and keeps the program code a little bit tidier.

first_function_example.py

```
def greet_me():
    print("Hello!")

def main():
    greet_me()

main()
```

Regarding to the order of statements in the example above:

- The call to the main() function must be placed after the definition of the main() function.
- The common convention is to place the main() function after all the other functions.

[27] Regarding to the terminology of Python: *Functions* are defined outside of class definitions. They are not associated with any objects. *Methods* are associated with the objects of the class they belong to.
[28] In a function call, Python interprets the parentheses as the *function call operator*.

9.2 Passing arguments to functions

The terms *parameter* and *argument* are often used to refer to the same thing. The **Python Language Reference** makes a clear distinction between these two terms. **Parameters** (*formal parameters*, *input parameters*) are the variables that we can define in the parameter list of a function. **Arguments** (*actual parameters*) are the values of the expressions in a function call.

Function call		*The header of the function definition*
`greet("Guido", "van Rossum")`	→	`def greet(first_name: str, last_name: str):`
Arguments		***Parameters***

If the function takes arguments, then a comma-separated parameter list should be enclosed within the parentheses in the function header. In the next example, the `greet()` function takes two arguments.

second_function_example.py

```
def greet(first_name: str, last_name: str):
    print(f"Hello {first_name} {last_name}!")

def main():
    greet("Guido", "van Rossum")

main()
```

The function body may contain one or more ***return*** *statements*[29]. When the function is called, its execution ends either at the end of the function body or when the execution comes to a return statement. In the third example, the `celsius_to_fahrenheit()` function takes one argument and returns one value.

third_function_example.py

```
def celsius_to_fahrenheit(celsius_degrees):
    return celsius_degrees * 9 / 5 + 32

def main():
    celsius = float(input("Enter celsius degrees: "))

    fahrenheit = celsiusToFahrenheit(celsius)
    print(f"{celsius}°C is {fahrenheit}°F")

main()
```

If we pass immutable arguments like ints, floats, strings or tuples to a function, the passing acts effectively like *call-by-value*.[30] When an argument is effectively passed by value in Python, we can imagine that the called function will operate on a copy of the original value. In the example below, we can imagine that the celsiusToFahrenheit() function operates on a copy of the original value (the inputted temperature value). That is, the celsiusToFahrenheit() function cannot change the original value.

```
celsius = float(input("Enter celsius degrees: "))
fahrenheit = celsiusToFahrenheit(celsius)
```

[29] (More advanced explanation) In Python, a function always returns something. If the function does not explicitly return anything, then the function implicitly returns a reference to a special single object of the type **None**.

[30] (More advanced explanation) In Python, the value to be passed to a function is always *a reference to an object*. In the example, when the celsius_to_fahrenheit() function is called, Python passes a reference to an object of type float to the function. Because we cannot change immutable objects in-place, the effect is much like making a copy.

Based on the computation in the celsius_to_fahrenheit() function, Python creates a new object of the float type. Finally, the celsius_to_fahrenheit() function returns a reference to the float object. Please notice that float objects are immutable.

Passing lists and other collections to functions

When we pass a list (or any other mutable collection) to a function, Python passes *a reference to the original list* to the called method. No copy of the original list is made. With a reference to the original list, the called function can access and change the content of the original list. This applies to all types of mutable collections.

In the example below, the function pad_with_zeros() appends zeros to the original list.

list_as_parameter.py

```
def pad_with_zeros(number_list: list, length: int):
    for i in range(len(number_list), length):
        number_list.append(0)

def main():
    numbers = [6, 1, 3]

    pad_with_zeros(numbers, 5)
    print(numbers)                # [6, 1, 3, 0, 0]

main()
```

Python binds parameter types when the function call is executed

Python is a ***dynamically typed*** language. That is, Python does not know about the type of the variable until the code is run. The variable is bound to a type when it is assigned a value.

In the example below, the print_max() function takes two arguments and prints the greater of them[31]. The function works fine with any two values that can be compared to each other with the '>' operator. If the operands are not valid for the comparison, the program crashes.

dynamically_typed_parameters.py

```
def print_max(a, b):
    if a > b:
        print(a)
    else:
        print(b)

def main():
    print_max(0, 3)                     # 3
    print_max(2.5, 1.25)                # 2.5
    print_max("Java", "Python")         # Python
    print_max([2, 0, 1], [2, 0, 8])     # [2, 0, 8]

main()
```

When Python compares two strings (s1 > s2), the result is based on the lexicographic order of the strings. When Python compares two lists (list1 > list2), it compares the lists element-by-element from left to right and determines the order based on the first non-matching element.

[31] Instead of the if statement, we can use a conditional expression as follows: `print(a if a > b else b)`

Positional arguments vs keyword arguments

When we are calling functions in Python, we can often choose between using ***positional arguments*** or ***keyword arguments*** (*named arguments*). A keyword argument consists of the argument name and value. Python matches the keyword argument by name with an input parameter in the function's parameter list. Sometimes we must use one or more keyword arguments. In the following function call, the first argument is a positional argument, and the second argument is a keyword argument: `print("Hello!", end=" ")`

Keyword arguments cannot appear before positional arguments in a function call. If there are more than one keyword argument, the order within the set of keyword arguments is not significant.

positional_vs_named_arguments.py

```
def divide(dividend, divisor):
    return dividend / divisor

def main():
    print(divide(10, 4))                    # 2.5
    print(divide(divisor=4, dividend=10))   # 2.5

main()
```

Default parameter values

When we define a function, we can specify a default value for each parameter. In the example below, the *message* parameter of the greet() function has a default value. Therefore, we can call the greet() method with either one or two arguments.

parameter_intial_value.py

```
def greet(first_name, message="Hello"):
    print(f"{message} {first_name}")

def main():
    greet("Harry")              # Hello Harry
    greet("Hermione", "Hi")     # Hi Hermione

main()
```

In the next example, the algorithm checks if the function is called with one or two arguments.

parameter_intial_value.py

```
def print_range(min: int, max: int = None):
    if max is None:
        max = min
        min = 1
    for x in range(min, max+1):
        print(x, end=" ")
    print()

def main():
    print_range(5)      # 1 2 3 4 5
    print_range(0, 3)   # 0 1 2 3

main()
```

9.3 Using type hints in a function definition

We can include ***type hints*** in function definitions to *document* the types of the arguments intended for the function and the type of the return value of the function.

type_hints_example.py

```
def add(a: int, b: int) -> int:
    return a + b

def main():
    x = add(2, 1.5) + "$"        # Note: This will fail!  We cannot concatenate an int and a str.
    print(x)

main()
```

Although Python does not allow invalid type hints, Python interpreter ignores them after validating their syntax. Instead, professional code editors can assist in writing a call to the function by showing the type hints as below[32].

```
4
5        (function) add: (a: int, b: int) -> int
6   x = add(2, 1)
```

In addition, there are third party tools that perform static type checking based on type hints. For example, the Pyright extension of the VS Code editor performs static type checking based on type hints and reports problems as below (before we run the program).

```
1  def add(a: int, b: int) -> int:
2      return a + b
3
4  def main():
5      x = add(2, 1.5) + "$"

PROBLEMS 2   OUTPUT   TERMINAL   JUPYTER   Filter (e.g. t
type_hint_example.py Funktiot 2
  Operator "+" not supported for types "int" and "Literal["$"]"  Pylance(reportGeneralTypeIssues)
    Operator "+" not supported for types "int" and "Literal["$"]"
  Argument of type "float" cannot be assigned to parameter "b" of type "int" in function "add"
    "float" is incompatible with "int"
```

In the example above, the type hints indicate that the add method expects to take two arguments of type int and its return value is of type int. The Pyright tool reports the following two problems on line 5: The second argument in the function call is of wrong type and Python does not allow an int to be joined with a string.

[32] Here is an ***advanced*** example: Based on the type hints, the code editor can show the programmer that the **fetch** function excepts the following two arguments: (1) a URL as a string and (2) a reference to a callback function that takes a string and an int as arguments and does not explicitly return anything.

```
4
5        (url: str, callback_function: (str, int) -> None) ->
         None
6
7   fetch()
```

9.4 Using the pass statement for a temporarily placeholder of required syntax

In Python, **pass** is a null operation. Nothing happens when Python executes it. It is useful as an explicit placeholder when a block of statements is required syntactically, but we want to add the final details later, or we do not want to remove a piece of code completely when it needs to be rewritten, or in some rare case, no code needs to be executed.

For example, here we want to complete the code of a function later:

```
def log_message_to_file(file, message: str):
    pass
```

Here we want to add the code later to handle a case of missing value:

```
if x is None:
    pass
```

9.5 Why create and use functions?

Below is a short list of some *generic reasons* why we create and use functions.

- To ***avoid undesired code duplication***
- To enable ***code reuse*** across multiple programs
- Improve ***testing***, ***maintenance,*** and ***internal quality of the software***

  A function can be separately tested with *automated unit tests*. The objective is to spend less time fixing defects. *Test-driven development* and *code refactoring* aim to improve the software's internal quality.

- ***Top-down programming*** and ***structuring the code***

  Functions allow you to *organize the logic into several smaller units*. Smaller units are *easier to design, write, test, understand, and maintain*. We can *start the design process from the top* and break a big problem down into smaller sub problems (smaller functions). Then, we can continue breaking these into several smaller functions until you get *down to very simple problems* to be solved. In addition, the work can be divided among a group of programmers who are specialized in different types of programming tasks.

- ***Abstraction*** and ***modularity***

  For example, from our viewpoint, the **`sort`** function is an *abstraction of a process* that takes an array as an input and sorts the array. In object-oriented programming, public functions provide the ***public interface*** (visible functionality) for an object. Private functions provide the ***internal implementation*** of the functionality. The objective is to hide implementation details from users of the function.

9.6 Distributing reusable functions in a library module

A ***library module*** is an ordinary Python source file containing functions etc. that can be used by any Python program.

In the example below, the file *calender_module.py* is a library module that contains a reusable set of general-purpose functions. Please notice that the library module is ***not*** a runnable Python program (nothing happens if we try to run it).

calendar_module.py

```
from datetime import datetime

def current_date():
    today = datetime.now()
    return f"{today.day}.{today.month}.{today.year}"

def current_time():
    timestring = str(datetime.now().time())
    return timestring[:8]

def is_leap_year(year):
    return (year % 4 == 0 and year % 400 != 0) or year % 100 == 0
```

The functions in the ***calendar_module.py*** file are used by the two Python programs below. To gain access to the functions of the library module, the programs must ***import the functions***. In this example, we assume that the library module and the programs are located in the same folder.

The first program explicitly imports[33] only the functions ***current_date***() and ***is_leap_year***() from ***calendar_module***. The program cannot use any other functions of ***calendar_module***.

program_one.py

```
from calendar_module import current_date, is_leap_year

print(current_date())

start_year = 2000
if is_leap_year(start_year):
    print(f"{start_year} is a leap year")
```

The second program explicitly imports only the functions ***current_date***() and ***current_time***() from ***calendar_module***.

program_two.py

```
from calendar_module import current_date, current_time

print(current_date())
print(current_time())
```

[33] In addition, there are the following two other variations of the **import** statement:

`from calendar_module import *` # => Any function from the module *calendar_module* can be used

`import calendar_module` # => Any function from the module *calendar_module* can be used by prepending the function name with the module name followed by a period. E.g. `calendar_module.current_date()`

In addition, we can give an alias name to an imported function as follows:

```
from calendar_module import is_leap_year as is_leap
```

9.7 Conditional execution of the main function

Sometimes we want to write such Python code that we can run it both as a script and import in another module. But when we import a module, Python immediately executes the main level code from the imported module. Therefore, we should control the execution of the code so that Python does not execute a certain part of the code when the code is imported.

When Python loads a Python source file, it sets a *global variable* named **__name__** and assigns **"__main__"** to the variable if we are running the source file as a script.

script_or_module.py

```
if __name__ == "__main__":
    print("This code is run as a script")
else:
    print("This code is imported as a module")
```

We can use **__name__** to control execution of our code. By using the if statement as below, we can allow Python to execute our **main**() function only if the code is run as a script. Otherwise, Python does not call the main() function.

```
if __name__ == "__main__":
    main()
```

In the example below, we can either import **my_module** or run it as a script without any issues.

When we run **my_module** directly, Python executes the **main**() function. The program prints **5.76**

my_module.py

```
def multiply(a, b):
    return a * b

def main():
    print(multiply(1.2, 4.8))       # 5.76

if __name__ == "__main__":
    main()
```

When we run **my_script** directly, Python loads also **my_module**. In **my_script**, we call the **multiply**() method of **my_module**. In this case, Python does not call the **main**() method of **my_module**, and the program prints **21**.

my_script.py

```
from my_module import multiply

print(multiply(3, 7))               # 21
```

NB! Without the if statement in **my_module**, Python would execute the **main**() method of **my_module** when it is loaded for importing the **multiply**() function, and the program would print both **5.76 and 21**.

9.8 How to write simple unit tests with doctest

We can use the **doctest module** to test code in a quick and easy manner by specifying an input and the correct output *in a docstring*. The doctest.**testmod**() function searches the docstring for pieces of text that look like interactive Python sessions, and then it executes those sessions to verify that they work exactly as shown.

In the next example, we have two functions in a module named ***math_routines***.

math_routines.py

```
def average(a, b):
    return (a + b) / 2

def multiply(a, b):
    return a * a
```

In an interactive Python interpreter session, we can import the module and call one of its functions:

```
>>> from math_routines import average, multiply
>>> average(2, 6)
4.0
```

In the ***test_math_routines*** script below, we have a *docstring* that includes four test cases for testing the above functions. The script calls the **testmod**() function that examines the docstring, executes the test cases, and prints a test report.

test_math_routines.py

```
"""
>>> from math_routines import average, multiply
>>> average(2, 6)
4.0
>>> average(2, 7)
4.5

>>> multiply(3, 3)
9
>>> multiply(2.5, 4)
10.0
"""
from doctest import testmod

testmod(verbose=False)          # verbose=True shows also passed test cases
```

When we run the ***test_math_routines*** script, it prints a test report that shows the failed test cases with the expected values and got values. Based on the test report, we can see that the **multiply**() function is not working correctly.

```
File "c:\Python codes\test_math_routines.py"
Failed example:
    multiply(2.5, 4)
Expected:
    10.0
Got:
    6.25

1 items had failures:
   1 of   5 in __main__
***Test Failed*** 1 failures.
```

10 List handling

10.1 Basic operations

List is a dynamic data structure. 'Dynamic' means that we do not have to specify the size of the data structure when we create it, and the structure can grow and shrink dynamically when we add new data in it or remove data from it[34].

A list is a *sequence of elements* in which an individual element is identified by its position in the list. To refer to a particular element in the list, we specify the name of the reference and the **index** (*position number*) of the element in the list. An index must be an integer (it can be any expression that evaluates to an integer)[35]. The values in a list can be of any type and do not have to be all of the same type.

We can create a list by writing its items as a comma-separated list enclosed in square brackets. If we do not write any items within the square brackets, the list will initially be empty.

```
lotto_numbers = [23, 12, 7, 9, 36, 2, 16]
```

In Python, the elements in a list are indexed from the front with ***positive indexes*** starting with zero[36] and from the back with ***negative indexes*** starting with -1.

lotto_numbers

0	1	2	3	4	5	6
23	**12**	**7**	**9**	**36**	**2**	**16**
-7	-6	-5	-4	-3	-2	-1

For example, we can access the last element in the list above either with the index value **6** or **-1**.

```
print(lotto_numbers[6])          # 16
print(lotto_numbers[-1])         # 16
```

We can use the for loop to ***iterate the list elements***.

```
for number in lotto_numbers:
    print(number, end=" ")       # The loop prints 23 12 7 9 36 2 16
```

We can ***print the list contents*** directly as below. The star (*****) operator unpacks the values from the list. The keyword argument ***sep*** adds a separator between the values when they are printed.

```
print(lotto_numbers)             # [23, 12, 7, 9, 36, 2, 16]
print(*lotto_numbers)            # 23 12 7 9 36 2 16
print(*lotto_numbers, sep=", ")  # 23, 12, 7, 9, 36, 2, 16
```

If we want to ***change the element values*** when we are iterating through the list, then we should keep track on the index value. The **len**() function returns the number of elements in the list.

list_iterate_with_an_index.py

```
points = [34, 52, 45, 57]

for index in range(len(points)):
    if points[index] > 50:
        points[index] = 50

print(points)                        # [34, 50, 45, 50]
```

[34] (Advanced remark) Python implements lists internally as *variable-length arrays* of references to other objects. List indexing has O(1) complexity.
[35] NB! The last element of a list is at the index equal to the ***count of items in the list minus one***.
[36] Technically speaking, a position number is the offset of the element in the list.

We can **add** new elements at the end of the list by calling the **append**() method.

list_add_elements.py

```
attendees = []

attendees.append("Frank")
attendees.append("Susan")
attendees.append("Alice")

print(*attendees, sep=", ")      # Frank, Susan, Alice

attendees.sort()
print(*attendees, sep=", ")      # Alice, Frank, Susan
```

We can **insert** a new element before an element of given index with the **insert**() method. All the previously existing elements starting from the given index are moved one position to the right in the list.

```
attendees.insert(0, "Guido")
```

We can **remove** an existing element of given index either with the **pop**() method (which also returns deleted value) or with the **del** keyword. The elements following the deleted item are moved one position to the left in the list.

```
attendees.pop(0)                    del attendees[0]
```

Python provides many useful functions and methods that we can apply on lists. In the next example, we ***sort*** the list in descending order, ***sort*** the list in ascending order, ***reverse*** the list, get the ***count of elements***, get the ***min*** value, get the ***max*** value, and get the ***sum*** of values.

list_handling.py

```
numbers = [3, 1, 4, 5, 9, 2, 6]

print(sorted(numbers))      # [1, 2, 3, 4, 5, 6, 9]
print(numbers)              # [3, 1, 4, 5, 9, 2, 6]

numbers.sort(reverse=True)
print(numbers)              # [9, 6, 5, 4, 3, 2, 1]

numbers.sort()
print(numbers)              # [1, 2, 3, 4, 5, 6, 9]

numbers.reverse()
print(numbers)              # [9, 6, 5, 4, 3, 2, 1]

print(len(numbers))         # 7
print(min(numbers))         # 1
print(max(numbers))         # 9
print(sum(numbers))         # 30
```

Please notice the following: The **sorted**() function creates *a new sorted copy* of the list[37]. The **sort**() method sorts the original list in place.

We can use the **in** operator to test if a value exists in a list.

list_membership_testing.py

```
numbers = [3, 1, 4, 5, 9, 2, 6]
print(5 in numbers)         # True
print(10 in numbers)        # False
print(10 not in numbers)    # True
```

[37] (Advanced remark) The sorted() function creates a *shallow* copy of the original list. To make a *deep copy* of a list or any other container, we can use the copy.deepcopy() function.

10.2 Using set() to remove duplicates from a list

Quite often Python makes it possible to write *neat one-liners*. That is, we can do several things on a single line of code without losing readability and overall cleanness of the code. For example, we can use more than one collection type in a single statement to achieve the desired result.

We can ***remove duplicates from a list*** using a **set**. Sets are *unordered* collections that **discard duplicate values**. Note: When we make a set out of another collection, the original order of items is not preserved.

remove_duplicates_from_a_list.py

```
numbers = [6, 6, 1, 33, 3, 5, 5, 5]

numbers = list(set(numbers))          # remove duplicates
print(numbers)                        # [1, 33, 3, 5, 6]
```

On the second line of the example above, we

1. Create a new set from the list by calling the **set**() function and passing a reference to the list as an argument. The **set**() function creates a new set, populates it with copies of values from the list discarding duplicates, and returns a reference to the set.
2. Create a new list from the set by calling the **list**() function and passing a reference to the set as an argument. The **list**() function creates a new list, populates it with copies of values from the set, and returns a reference to the list.

If we want to get a ***sorted list with no duplicates***, then we can use the **sorted**() method as below.

remove_duplicates_and_sort_the_list.py

```
numbers = [6, 6, 1, 33, 3, 5, 5, 5]

numbers = sorted(set(numbers))        # remove duplicates and sort the result
print(numbers)                        # [1, 3, 5, 6, 33]
```

We can ***join two lists*** with the '**+**' operator.

join_two_lists.py

```
a = ["cow", "dog", "cat", "bird"]
b = ["cat", "dog"]

print(a + b)                  # ['cow', 'dog', 'cat', 'bird', 'cat', 'dog']

print(list(set(a + b)))       # ['cow', 'dog', 'bird', 'cat']
print(sorted(set(a + b)))     # ['bird', 'cat', 'cow', 'dog']
```

If we want to ***count distinct values in a list***, then we can count the values in a **set** that we create from the list.

count_distinct_values.py

```
numbers = [2, 0, 2, 2, 1, 1, 2]

print(len(set(numbers)))      # 3
```

10.3 Comprehensive list of list operations

Below is quite comprehensive list of list operations. See also the Python documentation.
https://docs.python.org/3/library/stdtypes.html#sequence-types-list-tuple-range

List operations

Create a list	
`[]`	Creates an empty list
`[item1,item2,item3]`	Creates a list with some initial items
`[value] * n` `[0] * 100`	Creates a list of *n* items where each item is *value* Creates a list of 100 items where each item is zero
`list()`	Creates an empty list.
`list(iterable)`	Creates a list from any iterable data
`x.copy()`	Returns a shallow copy of the list[38].
Indexing	
`x[index]`	Returns the item at *index* or raises an error if *index* is out of bounds.
`x[index] = new_item`	Changes the item at *index* to *new_item,* or raises an error if *index* is out of bounds.
Slicing	
`x[start:stop]` `x[start:stop:step]`	Returns a ***slice*** from the list or raises an error if the slice is out of bounds.
`x[start:stop] = iterable` `x[start:stop] = []`	Replaces the slice with values from another collection or sequence Removes the slice from the list
Add and remove items	
`x.append(item)`	**Adds** a new item at the end of the list
`x.insert(index, item)`	**Inserts** a new item at the specified index
`x.pop(index)` `x.pop()`	**Removes** and also returns the item at the index, or the last item if no index is stated. Raises an error if *index* is out of bounds.
`x.remove(value)`	**Removes** the first occurrence of the value from the list or raises an error if the value does not exist in the list.
`x.clear()`	**Empties** the list
`x.extend(iterable)`	Adds each element of the iterable to the end of the list
`del x[index]`	Removes the ***item*** from the list or raises an error if *index* is out of bounds
`del x[start:stop]` `del x[start:stop:step]`	Removes the ***slice*** from the list or raises an error if the slice is out of bounds.
Statistics	
`len(x)`	Returns the number of items in the list
`max(x)`	Returns the largest item (if all items have the same data type)
`min(x)`	Returns the smallest item (if all items have the same data type)
`x.count(value)`	Returns the number of occurrences of the value in the list
`x.sum()`	Returns the sum of the items in the list or raises an error if the items are not summable.
Test membership, search a value	
`value in x`	True if *value* exists in the list
`value not in x`	True if *value* does not exist in the list

[38] Alternatively, we can get a shallow copy of a list by slicing the list as follows: `x[:]`
(Advanced info) We can use copy.**deepcopy**(x) to make a *deep copy* of a list (or any other type of data structure).

`x.index(`*value*`)`
 	
`x.index(`*value, index*`)`
	Returns the first index of the value in the list. or raises an error if the value does not exist in the list.
	Returns the first index of the value starting from the stated index.

Sort and reverse	See also: https://docs.python.org/3/howto/sorting.html#sortinghowto
`x.reverse()`	Reverses the list in place.

`x.sort()`
`x.sort(`reverse=**True**`)`
`x.sort(`**key**=*function*`)`
	Sorts the list in ascending order in place.
	Sorts the list in descending order in place.
	The ***key*** parameter specifies a function to be called on each list element prior to making comparisons. The function should take a single argument and *return a key to use for sorting purposes*.

`sorted(x)`
`sorted(x, reverse=True)`
`sorted(x, key=function)`
	Returns a ***sorted copy*** of the list. The original list is not changed.
	Returns a sorted copy of the list. The copy is sorted in descending order.
	The ***key*** parameter specifies a function to be called on each list element prior to making comparisons. The function should take a single argument and *return a key to use for sorting purposes*.

Join and repeat	
list1 `+` *list2* *list1* `+=` *list2*	Concatenates two lists Appends *list2* to *list1* in place
`x *` *n* `* x` `x *=` *n*	Returns the list repeated *n* times Repeats the list *n* times in place
Compare lists	
`<` `<=` `>` `>=` `==` `!=`	We can use the comparison operators to compare lists. Python compares individual items one by one from first item to last item. The comparison ends when the two items under examination are not equal or when there are no more items to be compared in either list.

Sorting strings in alphabetical order

When we are sorting strings that include ***special Finnish characters*** (ÅÄÖåäö), there are the following two issues to be tackled: (1) By default, "å" and "ä" are sorted in wrong order, and (2) all uppercase letters are placed before any lowercase letter. One way to solve these issues is to use the ***key*** parameter when we call the **sort**() method or the **sorted**() function.

sort_strings.py

```
def make_key(s: str) -> str:
    key = list(s)
    for i in range(0, len(key)):
        key[i] = key[i].lower()
        if key[i] == "ä":
            key[i] = "å"
        elif key[i] == "å":
            key[i] = "ä"
    return key

def main():
    strings = ["ä", "å", "Ö", "By", "bag"]

    print(sorted(strings))                # ['By', 'bag', 'Ö', 'ä', 'å']
    print(sorted(strings, key=make_key))  # ['bag', 'By', 'å', 'ä', 'Ö']

main()
```

10.4 List comprehension

List comprehension provides a concise syntax for generating a new list based on the content of an ***iterable collection*** (list, tuple, dictionary etc.) or ***iterable sequence*** (string, range etc.). The generalized syntax of list comprehension is as below. For each iteration of the for loop, the *expression* produces the value to be appended to the new list. The *expression* can be any Python expression[39].

```
[ expression for item in iterable ]
```

List comprehension can also include conditional logic.

```
[ expression for item in iterable if condition ]
```

In the example below, we use a normal for loop (no list comprehension yet).

```
price_list = [12, 40, 80]
prices_with_VAT = []

for price in price_list:
    prices_with_VAT.append(1.25 * price)

print(prices_with_VAT)                    # [15.0, 50.0, 100.0]
```

We can use *list comprehension* to write the code in a more concise way.

list_comprehension_1.py

```
price_list = [12, 40, 80]

prices_with_VAT = [price * 1.25 for price in price_list]

print(prices_with_VAT)                    # [15.0, 50.0, 100.0]
```

We can filter values if we write a condition just before the closing bracket.

list_comprehension_2.py

```
number_list = [3, -3, 1, 0, 4]
positive_numbers = [item for item in number_list if item > 0]

print(positive_numbers)                   # [3, 1, 4]
```

In the example below, we convert all strings in a list to uppercase.

list_comprehension_3.py

```
city_list = ["Oslo", "Helsinki", "Espoo"]
city_list = [city.upper() for city in city_list]

print(city_list)                          # ['OSLO', 'HELSINKI', 'ESPOO']
```

In the examples below, we iterate a range and a string.

list_comprehension_4.py

```
e = 2.718281828459045

print(*[round(e, i) for i in range(1, 5)])    # 2.7 2.72 2.718 2.7183
```

list_comprehension_5.py

```
text = "Monty Python tv program"
vowel_list = sorted(set([c for c in text.lower() if c in "aeiouy"]))
print(*vowel_list, sep=", ")              # a, o, y
```

[39] We can also nest list comprehensions.

```
names = [["john", "SMITH"], ["JILL", "jones"]]
names = [[s.capitalize() for s in item] for item in names]
print(names)  # [['John', 'Smith'], ['Jill', 'Jones']]
```

11 String handling

11.1 Overview

In Python, we can use operators, functions, and methods of the **str** class in string handling. We can compare strings with the normal comparison operators and extract substrings from a string with ***indexing*** and ***slicing***.

Here are some operators that we can use in string handling.

Operator	Description
`+`	String concatenation
`*`	String repetition; concatenates multiple copies of the same string
`[index]`	Indexing; get a character from the string. Returns a string of length one.
`[start:stop]`	Slicing: extract a substring from the string
`in`	Test if a substring exists in the string
`not in`	Test if a substring does not exist in the string

In addition, we can use generic built-in functions in string handling.

Function	Description
`len(s)`	Get the length of the string (the number of characters)
`max(s)`	Get the character with the highest character code
`min(s)`	Get the character with the lowest character code
`sorted(s)`	Convert the string to a sorted list

The example below shows some examples of different types of string handling operations.

string_handling_examples.py

```
s1 = "Python"
s2 = "Java"

print(s1 == s2)          # False              Normal comparison operators
print(s1 != s2)          # True
print(s1 > s2)           # True
print(s1 < s2)           # False
print(s1 + " 3.10")      # Python 3.10        String concatenation
print(5 * "Hi ")         # Hi Hi Hi Hi Hi     String repetition
print(14 * "-")          # --------------
print("ho" in s1)        # True               Using the in operator
print("ho" not in s1)    # False              Using the not in operator

text = "Example"
print(len(text))         # 7                  Using a function
print(text.upper())      # EXAMPLE            Using a method
print(text[0])           # E                  Indexing
print(text[-1])          # e                  Indexing with a negative index
print(text[2:5])         # amp                Slicing (extracting a substring)
for c in text:           #                    Iterating the string content
    print(c, end=" ")    # E x a m p l e
```

11.2 String handling with the methods of the str class

In Python, strings are ***immutable***. That is, the content of a string cannot be changed once it is created. When we want to modify an existing string or get a copy of one or more characters from the string, the function or method we call for this purpose ***creates and returns a new string***.

In Python, every data item is an object. An *object* combines data and ***methods*** that can act upon the object. A method is a function that can be accessed only through an object. We use the *dot operator* to call a method of an object. The following example shows how calling a method differs from calling a function:

```
s = "Java"
s = s.upper()     # Calling a method of a string object
x = len(s)        # Calling a built-in function
```

In the example below, we manipulate a string by calling several string methods.

strings_example_1.py

```
name = "  JAnNeS  "
name = name.strip()              # Remove leading and trailing space
name = name.lower()              # Convert to lowercase
name = name.replace("an", "o")   # Replace substrings
name = name.capitalize()         # Capitalise the first character and
                                 # convert the rest to lowercase
print(name)                      # Jones
```

```
# It is also possible to chain the function calls as below.
name = "  JAnNeS  "
name = name.strip().lower().replace("an", "o").capitalize()
print(name)                      # Jones
```

By calling a suitable string method, we can, for example,

- Trim the string (remove leading /trailing space)
- Convert the string to uppercase, lowercase, capitalized, titlecase etc.
- Replace substrings in the string with another substring
- Search a substring in the string
- Test characters (if they are digits, letters, alphanumeric, whitespace etc.)
- Split a delimited string around the delimiter into a list of substrings
- Convert a list of items into a single string where the items are separated by a specified separator

NB! All string methods and their arguments are explained in detail on
https://docs.python.org/3/library/stdtypes.html#string-methods

In the more advanced example below, we count distinct letters in a string.

count_distinct_letters.py

```
text = "The Pythons preferred spam, ham, and eggs"
letter_list = []
for c in text:
    if c.isalpha():
        letter_list.append(c.upper())
count = len(set(letter_list))              # set() removes duplicates
print(f"{count} different letters")        # 14
```

Here are some examples of the string methods.

Trim - remove leading and/or trailing space	
strip()	Remove leading and trailing space (or a specific set of characters)
lstrip()	Remove leading space (or a specific set of characters)
rstrip()	Remove trailing space (or a specific set of characters)
Convert case	
capitalize()	Capitalize the first character and convert the rest to lowercase
lower()	Convert the string into lowercase
upper()	Convert the string into uppercase
title()	Convert the first character of each word in the string to uppercase
Replace substrings	
replace(*old, new*)	Replace all occurrences of substring *old* by substring *new.*
Search or count substrings	
find(*substring, start, stop*)	Search for *substring* and return the ***first index*** of where it was found. With the ***optional*** *start* and *stop* we can tell at what index to start from and at what index to stop searching. Return **-1** if *substring* was not found.
rfind(*substring, start, stop*)	Search for *substring* and return the ***last index*** of where it was found.
count(*substring*)	Return the number of times *substring* occurs in the string
startswith(*prefix*)	Return true if the string starts with *prefix*
endswith(*suffix*)	Return true if the string ends with *suffix*
Check characters. Return true if all characters are...	
isalpha()	letter
isdigit()	digit
isalnum()	alphanumeric (letter or digit)
islower()	lowercase
isupper()	uppercase
isspace()	whitespace
Split	
split()	Split the string at the specified separator, and return a list of strings
splitlines()	Split the string at *universal* line breaks, and return a list of strings
Convert an iterable to a string	
join()	Convert the elements of an iterable (e.g., a list) into a single string

In the example below, we count letters and digits in a string.

character_counting.py

```
text = "Version 1.0A"
letter_count = 0
digit_count = 0

for c in text:
    if c.isalpha():
        letter_count += 1
    if c.isdigit():
        digit_count += 1

other_count = len(text) - letter_count - digit_count
print(f"{letter_count} letters")
print(f"{digit_count} digits")
print(f"{other_count} other characters")
```

11.3 Extracting parts of strings with indexing and slicing

In Python, strings, lists, tuples, and ranges are ordered, iterable sequences. We can use square brackets to reference parts of any sequences.

- **Indexing** allows us to access individual characters in a string directly. The first character in a string has the index **zero**. **Negative indexes** are relative to the end. The negative index of the last character in a string is **-1**.

  `s[index]` returns the character at the index as a new string.

- **Slicing**[40] allows us to extract a substring from a string. The two variants of the slice syntax are the following:

`s[start:stop]`	Returns a substring between *s*[***start***] and *s*[***stop*** **- 1**]
`s[start:stop:step]`	The ***step*** value sets the jump the slicing operation would take from one index to the other.

  In slicing, all the parameters are optional. The default *start value* is **0**, the default value *stop value* is the *length of the string*, and the default *step value* is **1**. **Slices** take characters **from the start index up to, but not including the stop index**. The ***step*** value sets the jump the slicing operation would take from one index to the other. A ***negative step value*** takes characters in reverse order from *s*[*start*] down to *s*[*stop + 1*].

For example, we can think about the indexing of the string "BROWN" as below.

0	1	2	3	4
B	**R**	**O**	**W**	**N**
-5	-4	-3	-2	-1

```
s = "BROWN"

print(s[0])          # B          Indexing
print(s[2])          # O
print(s[4])          # N

print(s[-5])         # B
print(s[-3])         # O
print(s[-1])         # N          Note: The last character is at the index -1

print(s[1:4])        # ROW        Slicing
print(s[2:])         # OWN        Up to the end of the string
print(s[:4])         # BROW       From the start of the string
print(s[0:4])        # BROW

print(s[0:5:2])      # BON        Slicing with a step
print(s[2::2])       # ON         Up to the end of the string
print(s[::2])        # BON        Every second character from s[0] up to s[s(len)-1]
print(s[::-2])       # NOB        Every second character from s[s(len)-1] down to s[0]
print(s[2::-1])      # ORB        Every character from s[2] down to s[0] [41]
```

If we omit both the start and stop index, then we get a ***copy of the whole string***.

```
print(s[:])          # BROWN
```

Slicing the whole string with the negative step **-1** gives us a ***reversed copy*** of the string.

```
print(s[::-1])       # NWORB      Every character from s[s(len)-1] down to s[0]
```

[40] NB! Slicing can be applied on any ordered sequence (e.g., string, list, tuple, range).
[41] (Advanced note) To set all parameters explicitly: `print(s[2:None:-1])`

11.4 Splitting and joining strings

The **split**() and **join**() methods are very valuable methods that are inverses of each other.

We can use the methods **split**() and **splitlines**() to turn a single string into a list of substrings.

- **`split`**`(`*`separator`*`)` splits the string ***at the specified separator***. If we specify no separator, then the string is split at whitespace. The separator is not included in the substrings.
- **`splitlines`**`()` splits the string ***at line breaks***. The line break is not included in the substrings. This method works in an OS-independent way with *universal newlines*. A terminal line break does not result in an extra line.

In the example below, we split strings at whitespace, commas, and line breaks.

split_strings.py

```
s1 = "Helsinki Stockholm Oslo"
s2 = "apple,orange,kiwi"

cities = s1.split()          # Split at whitespace
print(cities[0])             # Helsinki
print(cities[1])             # Stockholm
print(cities[2])             # Oslo

fruits = s2.split(",")       # Split at commas
print(fruits)                # ['apple', 'orange', 'kiwi']
```

```
text = "First line\nSecond line\nThird line"

lines = text.splitlines()    # Split at newlines
print(lines)                 # ['First line', 'Second line', 'Third line']
```

We can use the **join**() method to convert a list of strings (or an iterable collection) into a single string where the items are separated by a specified separator.

In the example below, we join a sequence of strings into one string with the **join**() method

join_strings.py

```
fruits = ["apple", "orange", "kiwi"]

separator = ", "
s1 = separator.join(fruits)
print(s1)                        # apple, orange, kiwi

s2 = ", ".join(fruits)
print(s2)                        # apple, orange, kiwi

s3 = " / ".join(fruits)
print(s3)                        # apple / orange / kiwi
```

In the example below, we sort the characters in a string to ascending order

sort_a_string.py

```
word = "compile"
s = "".join(sorted(word))
print(s)                # ceilmop
```

12 More list handling – Matrices

12.1 Creating and manipulating a matrix

Any types of values can be stored in lists. This means that a list can also contain other lists. For example, we can create a ***matrix*** (two-dimensional array) by nesting lists[42].

An example of a matrix is the game board of the tic-tac-toe game. The game board has three columns and three rows.

X	O	X
O	O	X
	X	O

We can represent the tic-tac-toe board as a matrix by nesting lists.

tictactoe_first.py

```
board = [["X", "O", "X"], ["O", "O", "X"], ["_", "X", "O"]]

for row in board:
    print(row)
```

The program prints the following:

```
['X', 'O', 'X']
['O', 'O', 'X']
['_', 'X', 'O']
```

We can print each matric cell separately by using nested for loops.

tictactoe_second.py

```
board = [["X", "O", "X"], ["O", "O", "X"], ["_", "X", "O"]]

for row in board:
    for column in row:
       print(column, end=" ")
    print()
```

The program prints the following:

```
X O X
O O X
_ X 0
```

In the example below, we count the number of occurrences of "X" in the ***first column*** of the matrix.

occurences_in_a_column.py

```
board = [["X", "O", "X"], ["O", "O", "X"], ["_", "X", "0"]]
count = 0

for row in board:
    if row[0] == "X":
        count = count + 1
print(f"{count} occurrences of X in the first column")
```

[42] This is not limited to two dimensions only. We can create n-dimensional arrays by nesting lists within lists. For example, we can represent the *Rubic's cube* by a 3-dimensional array.

In addition, we can use, for example, the NumPy library, which provides more tools for handling n-dimensional arrays. NB! In this material, the objective is to practice writing some algorithms on a basic matrix. Therefore, we do not use another Python library this time.

In the next example, we count the number of occurrences of "X" on the ***first row*** and the number of occurrences of "X" in the ***second column*** in a matrix. In this example, we have two reusable functions that we can use to count the number of occurrences of any item in any row or any column in any matrix.

matrix_counting_items.py

```
def count_item_on_row(matrix, row_index, item_to_count):
    count = 0
    for column in matrix[row_index]:        # Iterate through the desired row only
        if column == item_to_count:
            count = count + 1
    return count

def count_item_in_column(matrix, column_index, item_to_count):
    count = 0
    for row in matrix:
        if row[column_index] == item_to_count:
            count = count + 1
    return count

def main():
    board = [["X", "O", "X"], ["O", "O", "X"], ["_", "X", "0"]]

    x_on_row = count_item_on_row(board, 0, "X")
    print(f"{x_on_row} occurrences of X on the first row")

    x_in_col = count_item_in_column(board, 1, "X")
    print(f"{x_in_col} occurrences of X in the second column")

main()
```

12.2 Modifying items in a matrix

To change a cell in a matrix we should use both a ***row index*** and a ***column index***. The following assigns "_" to the cell in the *first column* of the *third row* in the matrix.

```
board[2][0] = "_"
```

In the following example, we assign "_" to all those cells in the matrix where the current value is not one of the following: "X", "O", or "_".

matrix_iteration_with_indexes.py

```
board = [["P", "O", "X"], ["a", "O", "X"], ["_", "X", "0"]]

for row in range(len(board)):
    for col in range(len(board[row])):
        if board[row][col] not in ("X", "O", "_"):
            board[row][col] = "_"

print(board)
```

The program prints the following:

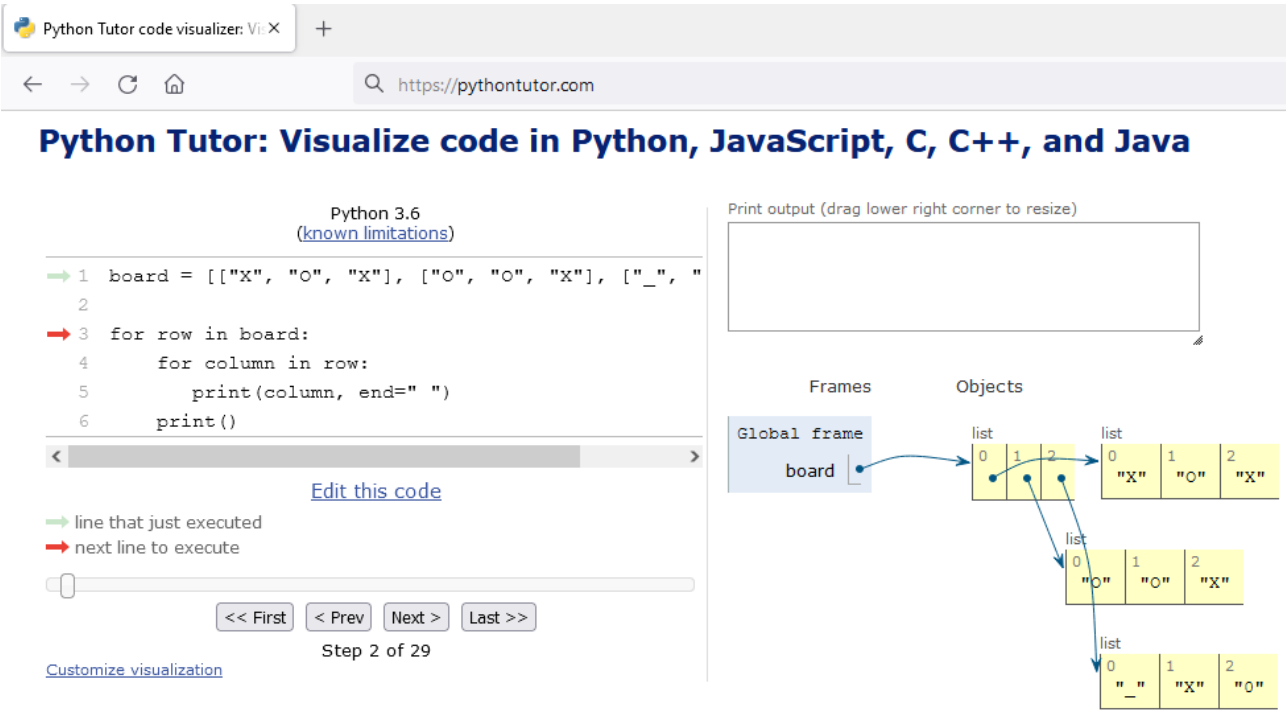
```
[['_', 'O', 'X'], ['_', 'O', 'X'], ['_', 'X', '_']]
```

12.3 Visualising a Python program with the Python Tutor tool

Philip Guo (University of Rochester) has created the interactive ***Python Tutor*** tool that we can use to visualise our Python programs[43]. This is a great tool especially for ***learning*** data structures and algorithms we use to manipulate data structures

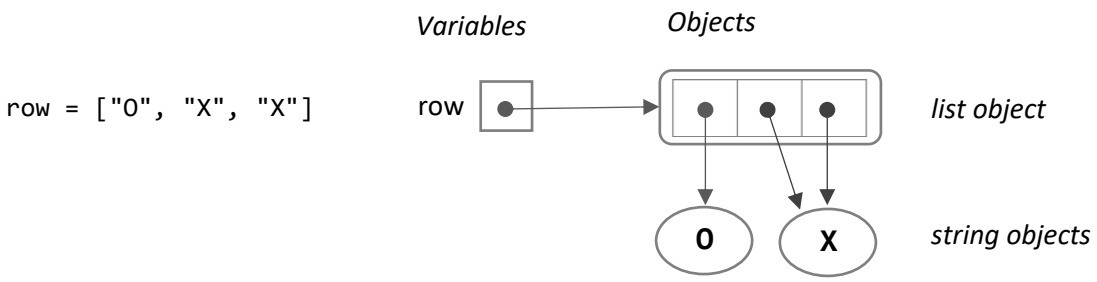
To use the tool, go to https://pythontutor.com/visualize.html, copy/paste your code to the text box, and click on "Visualize Execution".

The visualisation of our *tictactoe_second* program is as below. There we can see that the variable *board* refers to a list whose elements are referring to three other lists. We can animate the execution of the program step-by-step forward by clicking on "Next" and backward by clicking on "Prev". The tool shows the variables and the program output.



The tool ***shows references as arrows*** from the variable to the referenced object.

(More advanced visualisation) Please notice that *to keep the visualisation tidy*, the tool *takes a shortcut* and shows trivial data values (integers, strings etc.) as they were stored in the variables themselves. In reality, the actual data values are always stored in the referenced objects instead of variables. Below is a little bit more detailed visualisation.



Note: Python stores each distinct *literal value* (e.g., "X" and "O") only once. If we input values from the user, then Python stores each inputted value separately.

[43] (Additional advanced information) Philp Guo has also published an article about the tool.

13 References

13.1 Python handles all data items as objects

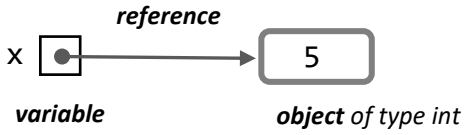
Python creates a variable when our code first assigns a value to the variable. Technically speaking, a variable is a memory slot, which can be identified with a name. In addition, a list is such a collection of variables that allows us to access each individual variable (element) in the list using an index instead of using a separate name for each element in the list.

Python handles all data items as objects. For example, when we assign 5 to a variable named x,

```
x = 5
```

Python

1. Creates an object of type int to represent the value of 5.
2. Creates the variable x if it does not exist yet.
3. Saves a link, which points to the object, to the variable x.



These links from variables to objects are called ***references***. Python handles data items by reference only. That is, ***an object itself is never stored in a variable***. When we assign an object to a variable, we are actually setting the variable to hold a ***reference***[44] to that object.

At the conceptual level, we can think that variables in Python are like hyperlink elements in an HTML document. In HTML, a hyperlink element contains a link to a web page or some other type of web resource. For example, here we have a link to "3.1 Objects, values and types" of the online Python language reference. We can access the actual data by following the link to the web page.

A variable never has any type information or constraints associated with it. Python finds the type of a data item by following the link from the variable to the object. Then, Python can determine how to apply - or restrict - the operation in which we are using the variable.

13.2 How to make a copy of a list or a collection of another type

When we are handling lists, sets, and dictionaries, we should be aware of how Python handles references.

In everyday programming, the fact that Python handles all data items as objects has real significance in the way how we write our code ***when we are handling mutable objects***, like **lists**, **sets** and **dictionaries**. For example, Python does not create a new list when we modify an existing list. We can assign references to the same list to any number of variables. This makes it possible to pass a reference to a list to a function, and the code in the function can modify the content of the list in place.

Sometimes we want to ***create a copy of a list*** or a collection of another type. The first thing to be noticed is that ***assignment statements never copy objects*** in Python.

[44] Behind the scenes, ***a reference is a pointer to the memory location at which the object is stored***. In diagrams, a reference is typically represented by an arrow from the variable to the object.

Assignment statements copy references only. They do not create new lists or other objects in Python.

Reference copy occurs when we copy a reference from one variable and assign the copied reference to another variable. The referenced object is ***not*** copied in this process.

two_variables_pointing_to_the_same_list.py

```
a = [1, 2]
b = a          # This copies only the reference (therefore, this is called reference copy)
               # That is, both a and b are pointing to the same list now
a.append(3)    # The list content becomes the following: [1, 2, 3]
b.append(4)    # The list content becomes the following: [1, 2, 3, 4]
print(a)       # [1, 2, 3, 4]
print(b)       # [1, 2, 3, 4]
```

We can make either a ***shallow copy*** or a ***deep copy*** of a list or a collection of another type.

⇨ If the list does ***not*** contain other lists or other collections, we make a ***shallow copy*** of the list.

shallow_copy.py

```
original_list = ["cat", "dog", "cow"]
new_list = original_list.copy()    # make a shallow copy

original_list.append("horse")
print(original_list)               # ["cat", "dog", "cow", "horse"]
print(new_list)                    # ["cat", "dog", "cow"]
```

Here are some different ways to make a shallow copy of a list:

```
b = a.copy()    # Call the copy method of the original list
b = a[:]        # Extract a slice that includes all items from the original list.
b = copy(a)     # from copy import copy and then call the copy function.
```

⇨ If the list ***contains other lists or other collections***, we can make a ***deep copy*** of the list to make the copy to be fully independent of the original list.

deep_copy.py

```
from copy import deepcopy

original_list = [["owl", "sparrow"], ["beagle", "poodle"]]
new_list = deepcopy(original_list)
```

In the case of ***nested lists***, the *shallow copy* makes a copy of the outer list only.

The *deep copy* makes copies of the outer list and all inner lists.

Why deep copy?

- The *shallow copy* does ***not*** copy nested inner lists (or other nested inner collections). Instead, it copies their references to the new list.
- The *deep copy* copies all nested lists (or other nested collections) recursively. This way, the new list will be fully independent of the original list.

Please see the difference in the example below. Use also https://pythontutor.com/visualize.html

deep_copy_vs_shallow_copy.py

```
from copy import deepcopy

original_list = [["owl", "sparrow"], ["beagle", "poodle"]]

shallow_list = original_list.copy()   # make a shallow copy
deep_list = deepcopy(original_list)   # make a deep copy

original_list[1][1] = "ELEPHANT"

print(original_list)       # [["owl", "sparrow"], ["beagle", "ELEPHANT"]]
print(shallow_list)        # [["owl", "sparrow"], ["beagle", "ELEPHANT"]]
print(deep_list)           # [["owl", "sparrow"], ["beagle", "poodle"]]
```

13.3 Loop counting is slower in Python (*for advanced learners*)

In the current set of Python's *built-in data types*, only ***lists***, ***sets*** and ***dictionaries*** are mutable. In Python, all the other data types are immutable. For example, if we want to increment an int by one, Python saves the result of the calculation as a reference to another object of type int.

Let us examine what happens when we first assign zero to a variable and then increment it by one. We can get the location where Python has saved an object using the **id**() function. Please notice that these locations are situation-specific. That is, if you run the code below, the locations will be different.

```
x = 0
print(x)        # 0
print(id(x))    # 2528718880976
x = x + 1       # Python saves to result of x + 1 as a new object and
                # saves a reference to the new object to the variable x
print(x)        # 1
print(id(x))    # 2528718881008
```

Let us examine what happens when we have a loop where we use a variable to count the number of repetitions. In contrast to many other popular programming languages, Python does not modify the counter value in place. Instead, Python creates a new object of type int each time we increment the counter value. This makes looping in Python slower than in many other programming languages.

incrementing_a_counter.py	**Output**

`i = 1`
`while i <= 4:`
`  print(id(i))`
`  i = i + 1`
	2894494171376
	2894494171408
	2894494171440
	2894494171472

How about looping a range with the for statement?

looping_a_range.py	**Output**
`for i in range(1, 5):` `  print(id(i))`	2559018991856 2559018991888 2559018991920 2559018991952

What will happen, if we have a vast range, like **range**(1_000_000)? Does the **range**() function really create one million int objects before the for statement iterates them? Nope. The **range**() function is a *generator function*, which creates int objects on demand, one-by-one. That is, when we iterate the range, the **range**() function creates only one new int object per each iteration.

Does the memory get very crowded with all these objects that we do not need after these loops have terminated? Nope. CPython performs automatic memory management. It keeps track on all objects and references in the memory, follows object allocations and object deallocations, and performs ***garbage collection*** when needed. In the garbage collection process, CPython can free the memory allocated for those objects that we cannot access any more.

14 Short overview of collection types (list, tuple, set, dictionary)

In Python 3.10, the built-in ***collection types*** are **list**, **tuple**, **set**, **frozenset** and **dict**.

Type	Example	Explanation
list	`numbers = [3, 1, 4, 5, 3, 5]`	A dynamic array of arbitrary items. Supports indexing and slicing.
tuple	`sizes = ("S", "M", "L", "XL")`	An immutable list
set	`country_codes = {42, 46, 47, 358}` `empty_set = set()`	An unordered set of unique, immutable items. Discards duplicate values.
frozenset	`frozenset_1 = frozenset({1, 2, 3})`	An immutable set
dict	`countries = {358: "Finland",` `46: "Sweden"}`	A dictionary[45] containing key-value pairs. Supports indexing by keys.

An instance of any collection type can usually contain references to instances of any type[46].

All built-in collections and sequences are ***iterable***. It is possible to *iterate them with the for loop.*

Lists, tuples, named tuples, strings, and ranges are ***sequences***. All sequences support *numeric indexing* and *slicing*.

We can usually *convert a collection or a sequence to another collection type*. For example,

number_list = **[**3, 1, 4, 1, 2, 3**]** number_set = **set**(number_list)	word = "Finland" letter_list = **list**(word)	a = **[(**358, "Finland"**)**, **(**46, "Sweden"**)]** country_dictionary = **dict**(a)

Below is a detailed summary of built-in collection types and sequences.

	Built-in collection types					*Other types*		
	list	**tuple**	**set**	**frozenset**	**dict**	**str**	**range**	**namedtuple**
Iterable; supports iteration `for item in collection:`	✓	✓	✓	✓	✓	✓	✓	✓
Sequence; supports numeric indexing and slicing	✓✓	✓✓	-	-	-	✓	✓	✓
Supports set operations such as union, intersect, difference	-	-	✓✓	✓✓	-	-	-	-
Indexed by keys	-	-	-	-	✓✓	-	-	-
Access values using field names	-	-	-	-	-	-	-	✓✓
Mutable (changeable)	✓	-	✓	-	✓	-	-	-
Ordered (insertion order)	✓	✓	-	-	✓[47]	✓	✓	✓
Duplicates allowed	✓	✓	-	-	-	✓[48]	-	✓
Can contain mutable items	✓	✓	-	-	✓[49]	-	-	✓

An **immutable collection** cannot be modified once it is created.
An **ordered collection** preserves its elements in insertion order.

[45] Python dictionaries are implemented internally as hash tables.
[46] Sets are exceptions here. The items in a set and frozenset must be immutable.
[47] Dictionaries are not ordered in some other programming languages. *dict* is ordered as of Python 3.7. There is also the older *collections.OrderedDict* type that is a subclass of the *dict* class.
[48] Duplicate characters are allowed in a string.
[49] In a dictionary, *keys* must be immutable values. (Advanced explanation) Python looks up items in *dictionaries* and *sets* by computing the item's location with a hash algorithm. Dictionaries and sets have O(**1**) search.

15 Tuples and sequences

15.1 Sequences (common features, comparison operations)

A ***sequence*** is a series of contiguous values. ***Strings***, ***ranges***, ***lists*** and ***tuples*** are sequences In the current set of built-in Python data types[50]. All sequences support **indexing** and **slicing** operations, membership testing (in, not in) etc[51].

sequences.py

```
my_string = "Hello!"
my_range = range(20, 50)
my_list = [2, 5, 8]
my_tuple = (2, 5, 8)

print(my_string[1])      # e
print(my_range[1])       # 21
print(my_list[1])        # 5
print(my_tuple[1])       # 5
```

A sequence instance may be ***compared to another sequence*** instance of *the same sequence type*. Python compares individual items one by one from first item to last item. The comparison ends when the two items under examination are not equal or when there are no more items to be compared in either sequence. Python applies *lexicographical ordering*.

comparing_sequences.py

```
print("ABC" < "X")                          # True
print([1, 2, 3] < [1, 2, 42])               # True
print([1, 2] < [1, 2, -5])                  # True
print(["C", "D"] < ["a", "b"])              # True
print([[1, 1], [2, 3]] < [[1, 1], [2, 2]])  # False
print([1, ("a", "b")] < [1, ("a", "c")])    # True
```

15.2 Tuples

The **tuple** is like an *immutable list*. That is, individual elements of a tuple cannot be added, changed, or removed after the tuple is created[52]. Otherwise, we can use tuples in the same way as we can use lists. Like lists, tuples can contain heterogeneous elements.

We can create a tuple by surrounding a comma-separated list of values with optional ***parenthesis***[53], or using the **tuple**() function.

tuple_example.py

```
my_tuple = (2, 5, 8)
print(my_tuple)            # (2, 5, 8)

x = 1
y = 2
another_tuple = (x, y)
print(another_tuple)       # (1, 2)

empty_tuple = ()
singleton_tuple = (42,)    # The comma is obligatory
```

[50] Since Python is an evolving language, other sequence data types may be added later.
[51] Common sequence operations: https://docs.python.org/3/library/stdtypes.html#typesseq-common
[52] Note: If the tuple contains a reference to a list (or any other mutable data structure), then we can change the content of the list. But we cannot remove or replace the reference in the tuple.
[53] NB! The use of parenthesis is recommended for clarity's sake.

Though ***tuples***[54] may seem similar to lists, they are often used in different situations and for different purposes. The shortlist below discusses some of these differences.

Why tuples?	
Technical differences between tuples and lists	• Tuples use parentheses instead of square brackets • Tuples are immutable • Tuples can be added to a set and used as a key in a dictionary • A tuple uses a little bit less memory than a list • In some operations, a tuple works slightly faster than a list • *Named tuples* support named fields. We can create a named tuple with the collections.namedtuple() function.
Python itself uses tuples for various purposes	• Sequence unpacking is quite often applied on tuples • Dictionary iterators return key-value pairs as tuples • The results of database queries are usually returned as a collection of tuples[55] • etc. etc.
Pragmatic use	Sometimes programmers use a *tuple* or a *named tuple* • When passing arguments to a function • When they want to return ***multiple values*** from a function • To act as a record with several fields (e.g., facts about a customer etc.). Especially, named tuples improve code readability. • Instead of creating their own data carrier class.

Creating a tuple is sometimes referred to as *packing a tuple*, because the values are "packed into" the tuple. Sequences can also be unpacked. In ***sequence unpacking***, we can assign the values in the sequence to several variables with a single assignment statement.

sequence_unpacking.py

```
my_tuple = (3, 1, 4)

x, y, z = my_tuple            # Note: sequence unpacking
print(f"{x} {y} {z}")         # 3 1 4
```

We can **unpack** any types of sequences. Please notice that the number of variables ***must*** match to the number of elements in the sequence.

more_unpacking.py

```
x, y = [3, 1]
c1, c2, c3 = "ABC"

print(f"{c1} {c2} {c3}")       # A B C
```

In the example below, we have a Python one-liner to ***swap two values without using a temporary variable***. First, Python packs the comma-separated list of values on right-hand side of the assignment operator into a tuple. Then, Python unpacks the tuple to two separate elements in the list.

swap_two_values.py

```
a = [3, 1, 4]

a[0], a[1] = a[1], a[0]       # Create a tuple and unpack it
print(a)                      # [1, 3, 4]
```

[54] The term 'tuple' originates as an abstraction of the sequence where words are ending in "-tuple": single, double, triple, quadruple, quintuple, sextuple, septuple, octuple... The prefixes are taken from the Latin names of the numerals.

[55] The term 'tuple' is also included in the relational data model. In a database, a tuple is realised as a single record/row.

greatest_common_divisor.py *(Advanced example. Note: This is **not** very readable code)*

```
def greatest_common_divisor(x, y):
    while x % y:
        x, y = y, x % y    # NB! Python computes x % y when it creates the tuple
    return y

def main():
    print(greatest_common_divisor(12, 16)     # 4
    print(greatest_common_divisor(28, 98))    # 14

main()
```

In the next example, we the **divmod**() function computes both the quotient and the remainder of a floor division. The function returns these two values in a tuple. The **main**() function calls the **divmod**() function and gets the quotient and remainder subscripting the tuple.

divmod_version_1.py

```
def divmod(divident, divisor):
    q = divident // divisor
    r = divident % divisor
    return (q, r)

def main():
    my_tuple = divmod(11, 3)
    print(my_tuple[0])                        # 3
    print(my_tuple[1])                        # 2

main()
```

Instead of subscripting a tuple, we can use ***sequence unpacking*** as below to improve readability of the code. In addition, it is possible to omit the parentheses when we create a tuple.

divmod_version_2.py

```
def divmod(divident, divisor):
    q = divident // divisor
    r = divident % divisor
    return q, r                               # Note: we create a tuple here!

def main():
    quotient, remainder = divmod(11, 3)       # Note: sequence unpacking!
    print(quotient)                           # 3
    print(remainder)                          # 2

main()
```

Please notice that it just looks like the *return statement* in the **divmod**() function is returning several values. The fact is that ***a return statement can return only one value***. In the above example, the return statement returns *a reference to a tuple*, which contains two items. We just should know that Python performs sequence unpacking automatically.

While tuples and named tuples are immutable, *the values they store do not have to be immutable*. In the example below, we store the person's name and her programming language qualifications in a tuple. The names of programming languages are stored in a list. That is, we can modify the content of the list after we have created the tuple.

list_in_a_tuple.py

```
p = ("Susan", ["JavaScript"])
print(p[1])                              # ['JavaScript']

p[1].append("Java")
p[1].append("Python")

print(p[1])                              # ['JavaScript', 'Java', 'Python']
```

15.3 Named tuples

We can use the **namedtuple**() function to create an immutable, tuple-like data structure with ***field names***. We can access the values in a named tuple using the dot notation and the field names. The **namedtuple**() function is available in Python's ***collections*** module.

Below the namedtuple() function takes two arguments. The first argument is the name of our named tuple type. The second argument is a string that includes the field names. We can separate the field names by commas or space. The order of the field names is significant.

named_tuple_example.py

```
from collections import namedtuple

Point = namedtuple("Point", "x, y, color")

p1 = Point(x=3, y=4, color="black")
print(f"{p1.x}, {p1.y}: {p1.color}")      # 3, 4: black

p2 = Point(5, 6, "blue")
print(f"{p2.x}, {p2.y}: {p2.color}")      # 5, 6: blue
```

Below is a new version of the previous "list in a tuple" example. This time, we use a named tuple instead of an ordinary tuple.

list_in_a_named_tuple.py

```
from collections import namedtuple

Person = namedtuple("Person", "name, languages")

p = Person(name="Susan", languages=["JavaScript"])

print(p.languages)                       # ['JavaScript']

p.languages.append("Java")
p.languages.append("Python")
print(p.languages)                       # ['JavaScript', 'Java', 'Python']
```

Please notice the following differences in readability between a *namedtuple* and a *tuple*:

p.**languages** tells us explicitly what fact we are accessing. This is position-independent.

p**[1]** gives us no idea what fact we are accessing. The position of the item is significant.

17 Dictionaries

A **dictionary** is a collection that stores items as ***key–value pairs.*** The items in a dictionary are ***indexed by keys***[56]. *Keys* must be unique within the dictionary. Keys can be of any *immutable* type. *Values* can be of any type of data.

Selected dictionary operations	https://docs.python.org/3/library/stdtypes.html#typesmapping
`{}`	Creates an empty dictionary
`{key1: value1, key2: value2}`	Creates a dictionary with some initial items (key-value pairs)
`d[key]`	Returns the value associated to *key* or raises an error if *key* is not found
`d[key] = new_value`	**Adds** the item *key:new_value* to the dictionary. Or, if *key* already exists in the dictionary, **changes** the value associated to *key* to new_*value*.
`key in d`	True, if *key* exists in the dictionary
`key not in d`	True, if *key* does not exist in the dictionary
`len(d)`	Returns the number of items (key-value pairs) in the dictionary
`max(d)`	Returns the largest key value (if all keys have the same data type)
`min(d)`	Returns the smallest key value (if all keys have the same data type)
`list(d.items())`	Returns a list of the items in the dictionary. Each item in the list is a tuple of the format (*key*, *value*)
`list(d.keys())`	Returns a list of the keys in the dictionary
`list(d.values())`	Returns a list of the values in the dictionary
`d.clear()`	Empties the dictionary
`d.get(key)`	Returns the value associated to *key* or **None** if *key* is not found
`d.pop(key)` `d.pop(key, default)`	Removes the key-value pair from the dictionary and returns the value associated to the key, or raises an error if *key* is not found *default* sets the value which pop() returns if the item is not found
`del d[key]`	Removes the item from the dictionary or raises an error if *key* is not found

We can ***create a dictionary*** by writing its items as a comma-separated list enclosed in ***braces***. If we do not write any items within the braces, the dictionary will initially be empty.

dictionary_example.py

```
acronyms = {"AM": "before noon", "PM": "after noon"}

print(acronyms["PM"])      # after noon
print(acronyms["AM"])      # before noon
```

We can ***add a new item*** to a dictionary using the new key as the index when we assign the value to the dictionary.

country_code_dictionary.py

```
country_codes = {}

country_codes["Finland"] = 358
country_codes["Sweden"] = 46
country_codes["Norway"] = 47

print(country_codes["Sweden"]) # 46
print(country_codes)           # {'Finland': 358, 'Sweden': 46, 'Norway': 47}
```

[56] Python implements dictionaries as resizable hash tables. A hash table uses a hash function to compute an index into an array from which the desired value can be found. Note: It is usually unwise to use floats as dictionary keys since Python stores them as approximations.

As of Python version 3.7, dictionaries are ordered. That is, we can iterate the items in a dictionary in their insertion order. In Python 3.6 and earlier, dictionaries are unordered.

In the example below, we input country names and country codes from the user until the user enters an empty string as country name. Note: an empty string evaluates to *false* in a condition.

input_and_save_data_to_a_dictionary.py

```
country_codes = {}

name = input("Enter country name: ")
while name:
    code = int(input("Enter country code: "))
    country_codes[name] = code
    name = input("Enter country name: ")

print(country_codes)
```

We can ***replace an existing item*** using a key that already exists in the dictionary when we assign a new value. We can ***remove an existing item*** (key-value pair) using the **pop**() method (which also returns the value associated to the key) or the **del** keyword.

grade_words.py

```
grades = {1: "Fair", 3: "Nice", 5: "Excellent", 42: "Level 42"}

grades[3] = "Good"
grades.pop(42)            # alternatively: del grades[42]

print(grades)             # {1: 'Fair', 3: 'Good', 5: 'Excellent'}
```

Accessing a non-existent dictionary key results in a **KeyError**. We can avoid this error either using the **in** operator to check if the key exists in the dictionary or using the **get**() method. The **get**() method takes a key as argument and returns the corresponding value. If that key is not found, **get**() returns **None**.

avoid_key_error.py

```
grades = {1: "Fair", 3: "Good", 5: "Excellent"}

if 88 in grades:
    print(grades[88])
else:
    print("88 is not found in the dictionary")

grade_text = grades.get(42)
if grade_text:
    print(grade_text)
else:
    print("42 is not found in the dictionary")
```

We can use the **items**() method when we want to ***traverse the entire contents of a dictionary***. This way, we get the key-value pairs one-by-one as *tuples*, where the first item is the key, and the second item is the value. Python automatically *unpacks the tuple* if we write two comma-separated variable names between '**for**' and '**in**'. That is, Python assigns the tuple's first item to the first variable and the tuple's second item to the second variable.

iterate_dictionary.py

```
acronyms = {"AM": "before noon", "PM": "after noon"}

for key, value in dictionary.items():
    print(f"{key} means {value}")       # AM means before noon
                                        # PM means after noon
```

In the following example, we count the number of occurrences of each word in a list. To iterate dictionary items in sorted order, we can use the **sorted**() function.

word_counting.py[57]

```
wordlist = ["bat", "dog", "dog", "cat", "dog"]
dictionary = {}

for word in wordlist:
    if word in dictionary:
        dictionary[word] += 1
    else:
        dictionary[word] = 1

for word, count in sorted(dictionary.items()):
    print(f"{word} {count}")                    # bat 1  cat 1  dog 3
```

The **keys**() method returns an iterator over the keys of the dictionary. The **values**() method returns an iterator over the values of the dictionary. The **items** () method returns an iterator over the items of the dictionary. The iterator returns copies of the items as tuples of the form (*key*, *value*).

keys_and_values.py

```
country_codes = {'Finland': 358, 'Sweden': 46, 'Norway': 47}

print(list(country_codes.keys()))       # ['Finland', 'Sweden', 'Norway']
print(sorted(country_codes.keys()))     # ['Finland', 'Norway', 'Sweden']

print(list(country_codes.values()))     # [358, 46, 47]
print(sorted(country_codes.values()))   # [46, 47, 358]

print(list(country_codes.items()))      # [('Finland', 358), ('Sweden', 46), ('Norway', 47)]
```

In the example below, we ***nest dictionaries***. We save facts about each course to its own dictionary. These dictionaries we add to the 'courses' dictionary.

nested_dictionaries.py

```
courses = {}

courses["C002"] = {"code": "C002", "name": "Python programming", "credits": 5}
courses["C003"] = {"code": "C003", "name": "Java programming", "credits": 5}

course_code = "C003"
if course_code in courses:
    course = courses[course_code]
    print(course["code"])               # C003
    print(course["name"])               # Java programming
    print(course["credits"])            # 5
else:
    print("Unknown course code")
```

[57] (Advanced details) Here are two more ways to solve the word counting problem. We can create the dictionary directly from a list with the *dict.fromkeys*() method, or we can use the *Counter* class from the *collections* module as below.

```
wordlist = ["bat", "dog", "dog", "cat", "dog"]

dictionary = dict.fromkeys(wordlist, 0)
for word in wordlist:
    dictionary[word] += 1

for word, count in sorted(dictionary.items()):
    print(f"{word} {count}")
```

```
from collections import Counter

wordlist = ["bat", "dog", "dog", "cat", "dog"]

counter = Counter(wordlist)

for word, count in sorted(counter.items()):
    print(f"{word} {count}")
```

In the example below, we input facts about countries from the user until the user enters an empty string as country name. Finally, we input one more country name from the user, get the corresponding item from the dictionary, and print the name of the capital of the country.

input_data_from_the_user_and_save_it_to_a_dictionary.py

```
countries = {}

country_name = input("Enter country name: ")
while country_name:
    population = int(input("Enter population: "))
    capital = input("Enter capital: ")

    countries[country_name] = {"country name": country_name,
                               "population": population,
                               "capital": capital}
    country_name = input("Enter country name: ")

print()

country_name = input("Country to be searched: ")
if country_name in countries:
    capital = countries[country_name]["capital"]
    print(f"{country_name}'s capital is {capital}")
else:
    print("Unknown country")
```

In the example above, we create a separate dictionary for each country and add these dictionaries as the values to the *countries* dictionary.

18 Reading text files

18.1 Reading the content of a text file

A ***text file*** contains ***plain text*** without any formatting. That is, we can open a text file in any text editor. For example, program source codes are saved as text files. A text file is internally structured as a ***sequence of lines***. The ***newline*** ('**\n**') signifies the end of a line of text and the start of a new one.

In Python, there are several different ways to read and write text files[58]. In this material, we focus on using the **Path** class from the **pathlib** module to avoid problems with the following: locating the file to be accessed, differences in folder path separators, and differences in character encoding.

Text file access using the Path class - some basic operations	
Path(*path_to_the_file*)	Returns an object that contains the absolute path to the file
read_text(*encoding*)	Reads the whole content of an existing text file and returns it as a single string. *encoding* sets the character encoding of the file. We are on safe ground when we set it as follows: `encoding="UTF-8"`
write_text(*string*, *encoding*)	Writes a string to a text file. If the file already exists, this replaces the previous content of the file with new content. If the file does not exist yet, this creates a new file.

The first program writes some text to a text file named "test.txt".

write_text_to_a_file.py

```
from pathlib import Path

path = Path("test.txt")
path.write_text("Monty Python's Flying Circus", encoding="UTF-8")
```

The second program reads the content of a text file named "test.txt" as a single string and prints the string.

read_text_from_a_file.py

```
from pathlib import Path

path = Path("test.txt")
contents = path.read_text(encoding="UTF-8")        # Read the file content as a single string
print(contents)
```

The output of the program is as below.

```
Monty Python's Flying Circus
```

Note 1: In the above example, Python saves the file "text.txt" to the current working folder. The working folder is not necessarily the same folder where the current script is saved. In Python, you can see current working folder's name as below.

```
import os
print(os.getcwd())
```

Note 2: We can open the text file "test.txt" in any text editor.

Note 3: To practice with text file reading we can create new text files in a text editor.

[58] Here are two other ways to read the content of a text file.

```
with open("test.txt", "r", encoding="UTF-8") as infile:
    content = infile.read()
print(content)
```

```
infile = open("test.txt", "r", encoding="UTF-8")
content = infile.read()
infile.close()
print(content)
```

We can handle the *'file not found error'* as follows:

```
try:
    path = Path("somefile.txt")
    # ...
except FileNotFoundError:
    print("The file is not found")
```

18.2 Using Path to locate the absolute path to a folder

The script which we are executing is not always saved to the current working folder.

To access a text file that is saved in the *same folder as the current script* we should determine the ***absolute path*** to the text file. The absolute path should include all folders, starting from the root folder, which lead to the file.

We can use the services of the **Path** class as below to get the current script's folder path. Then, we can append the name of a text file to the path.

```
filename = "notes.txt"
path = Path(__file__).resolve().parent     # 1. Get the current script's full folder path
path = path / filename                     # 2. Append filename to the folder path
```

In the context of a Path object, '**/**' acts as a special operator[59] which can be used to append the name of a file to the path. The built-in variable **__file__** holds the full path to the current script, including the file name.

Suppose that a text file called *notes.txt* contains the following three lines of text:

```
Python is named after the
Monty Python's Flying Circus,
a BBC comedy series from the 1970s
```

The program below reads the content of the text file and prints the lines with line numbers.

read_file_content_and_print_it.py

```
from pathlib import Path

filename = "notes.txt"
path = Path(__file__).resolve().parent        # 1. Get the current script's full folder path
path = path / filename                        # 2. Append the file name to the folder path

contents = path.read_text(encoding="UTF-8")   # 3. Read the file content as a single string
lines = contents.splitlines()                 # 4. Split the string into a list of strings

line_number = 0
for line in lines:                            # 5. Iterate through the list
    line_number += 1
    print(f"{line_number}: {line}")           # 6. Print each line with a line number
```

The output of the program is as below.

```
1:  Python is named after the
2:  Monty Python's Flying Circus,
3:  a BBC comedy series from the 1970s
```

[59] (Advanced explanation) Python supports *operator overloading*. That is, in a class definition, we can specify how certain operators should work with objects created from the class.

Alternatively, we can use the **enumerate**() function to create the line numbers.

```
for line_number, line in enumerate(lines, start=1):
    print(f"{line_number}: {line}")
```

18.3 Reading a CSV file

A ***CSV file*** (*comma-separated values file*) is a text file where ***each line represents a data record***. Each line consists of one or more fields, typically separated by either comma or semicolon (or some other character). The content of a CSV file can represent tabular data. Sometimes the first line of the CSV file contains the column names.

CSV is a common format for easy data interchange between different systems. For example, Excel and many database management systems can import and export data in the CSV format. CSV files are also commonly used in data science.

Suppose that a CSV file called *courses.**csv*** contains the following facts about courses: *course codes*, *course names* and *credits*. In our example, the file contains three lines of text as below.

```
CS03;Python Programming;5
CS04;Introduction to Databases;3
CS05;Java Programming;5
```

On each line, there is a semicolon-separated list of three values.

The program below reads the content of the "courses.csv" file as a single string and splits the string into a list of strings. Then, the program iterates through the list and splits each line around the field separator (';') to access each individual value.

reading_a_csv_file.py

```
from pathlib import Path

filename = "courses.csv"
path = Path(__file__).resolve().parent          # 1. Get the current script's folder path
path = path / filename                          # 2. Append the file name to the folder path

contents = path.read_text(encoding="UTF-8")     # 3. Read the file content as a single string
lines = contents.splitlines()                   # 4. Split the string into a list of strings

sum = 0
for line in lines:                              # 5. Iterate through the list
    fields = line.split(";")                    # 6. Split the line into a list of strings
    print(f"{fields[0]}: {fields[1]}, {fields[2]} credits")
    sum = sum + int(fields[2])

print(f"{sum} credits in total")
```

The output of the program is as below.

```
CS03: Python Programming, 5 credits
CS04: Introduction to Databases, 3 credits
CS05: Java Programming, 5 credits
13 credits in total
```

List of Python keywords used in the examples

Python 3.11 keywords and soft keywords

False	**break**	**except**	*is*	**return**
None	*case*	**finally**	*lambda*	**try**
True	*class*	**for**	*match*	**while**
and	*continue*	**from**	**nonlocal**	**with**
as	**def**	**global**	**not**	*yield*
assert	**del**	**if**	**or**	
async	**elif**	**import**	**pass**	
await	**else**	**in**	*raise*	

The bolded keywords are used in the examples in this document.

Note: In Python 3.10, *case* and *match* are soft keywords. That is, they can syntactically act as keywords in contexts related to the pattern matching. In other contexts, Python treats them as normal identifiers. NB! The best practice is avoid using them as identifiers in our programs.